



# Chapter 6

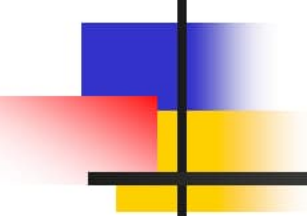
## THE TRANSPORT LAYER

# Contents



1. Introduction
2. The Physical Layer
3. The Data Link Layer
4. The Medium Access Control Sub-Layer
5. The Network Layer
6. **The Transport Layer**
7. The Application Layer
8. Network Security

|                       |           |
|-----------------------|-----------|
| Chapter 7 Application |           |
| Chapter 6 Transport   |           |
| Chapter 5 Network     |           |
| Chapter 3 LLC         | Data Link |
| Chapter 4 MAC         |           |
| Chapter 2 Physical    |           |



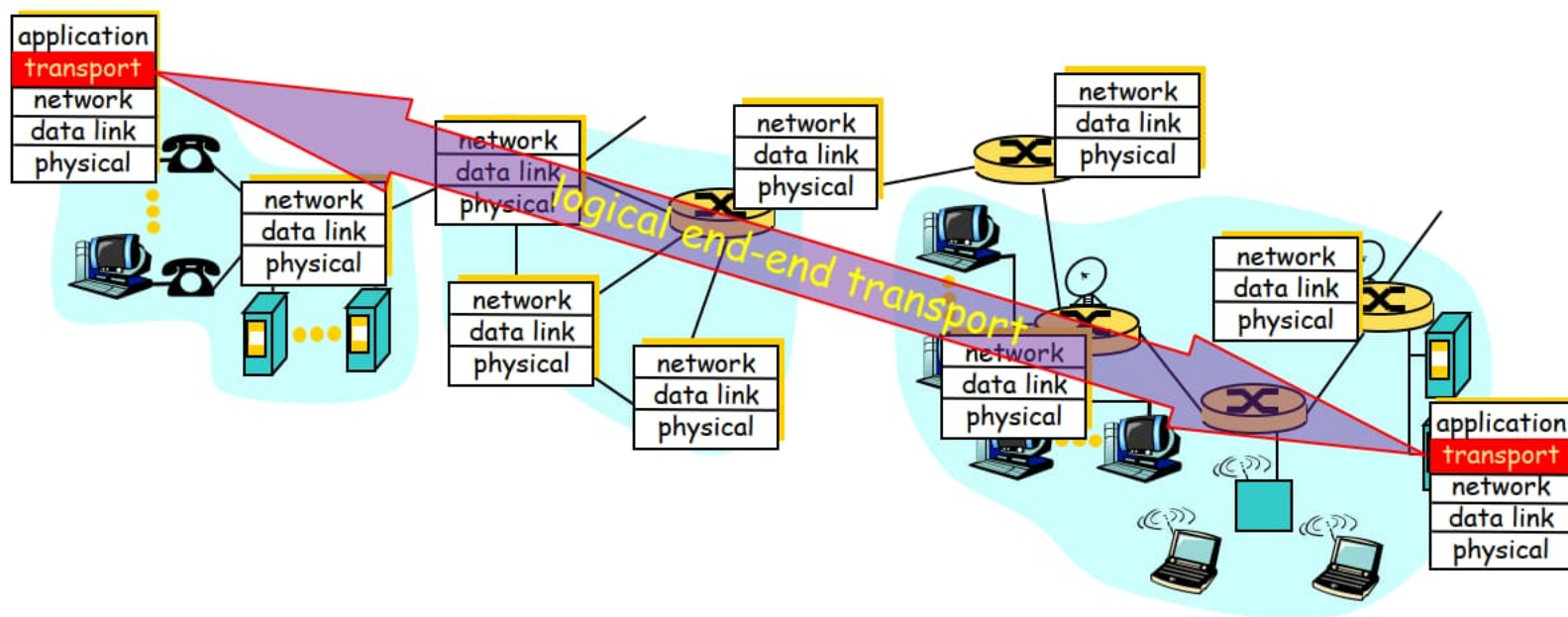
# Contents

---



1. The Transport Service
2. Elements of Transport Protocols
3. Congestion Control
4. The Internet Transport Protocols: UDP
5. The Internet Transport Protocols: TCP
6. Performance Issues
7. Delay-Tolerant Networking

# 6.1 The Transport Service

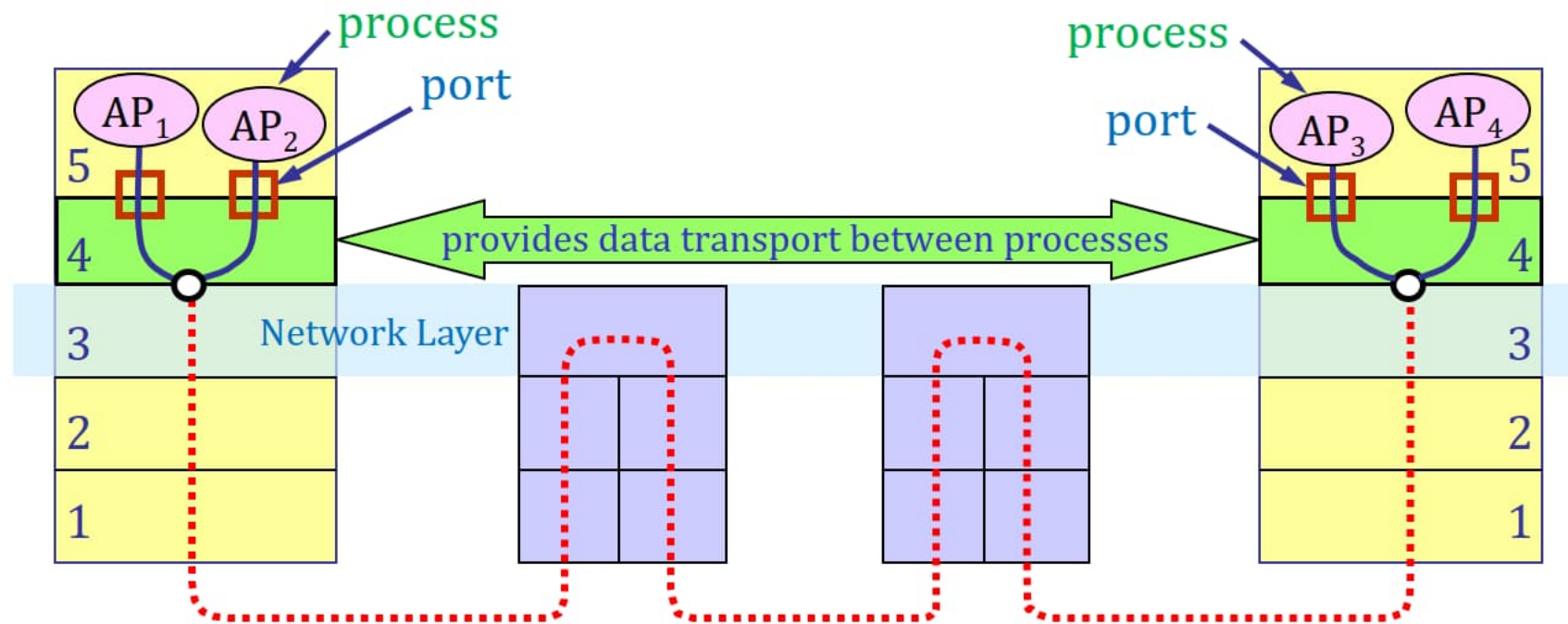


## ➤ Transport Layer

- ✦ Together with the **network** layer, the **transport** layer is the **heart** of the protocol hierarchy.
- ✦ **End to end** protocol. 端点-端点



# Communicates between machines



- The transport layer provides data transport from a process on a source machine to a process on a destination machine.

- ✦ Multiplexing 复用
- ✦ De-multiplexing 解复用

## 6.1.1 Services Provided to the Upper Layers

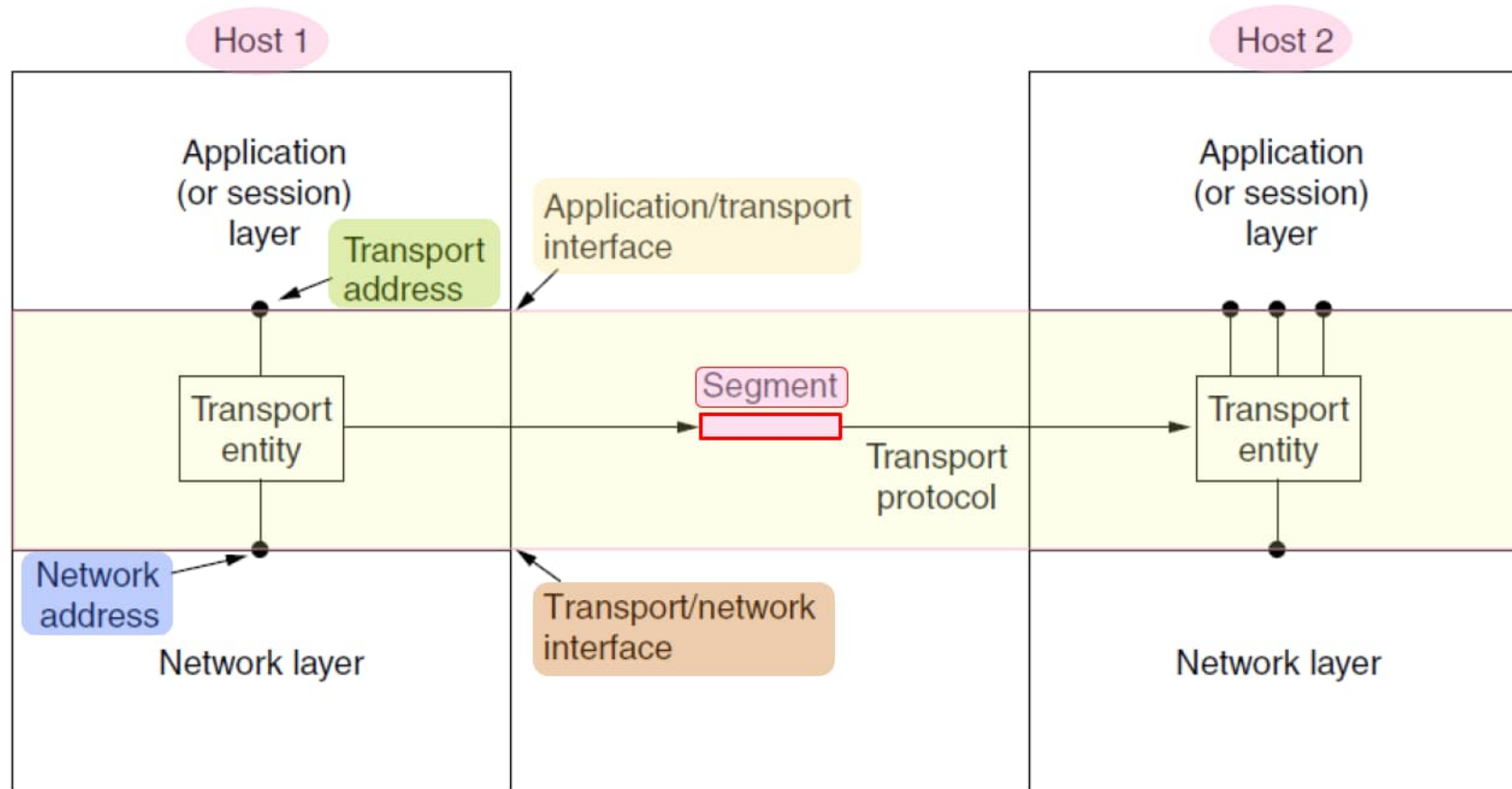


Figure 6-1. The network, transport, and application layers.

## 6.1.2 Transport Service Primitives

### ➤ Transport Service Primitives 传输层服务原语

| Primitive  | Packet sent        | Meaning                                    |
|------------|--------------------|--------------------------------------------|
| LISTEN     | (none)             | Block until some process tries to connect  |
| CONNECT    | CONNECTION REQ.    | Actively attempt to establish a connection |
| SEND       | DATA               | Send information                           |
| RECEIVE    | (none)             | Block until a DATA packet arrives          |
| DISCONNECT | DISCONNECTION REQ. | This side wants to release the connection  |

Figure 6-2. The primitives for a simple transport service.

# Transport Protocol Data Unit



➤ **TPDU: Transport Protocol Data Unit**      传输层协议数据单元

✦ **Segment**    (数据)段

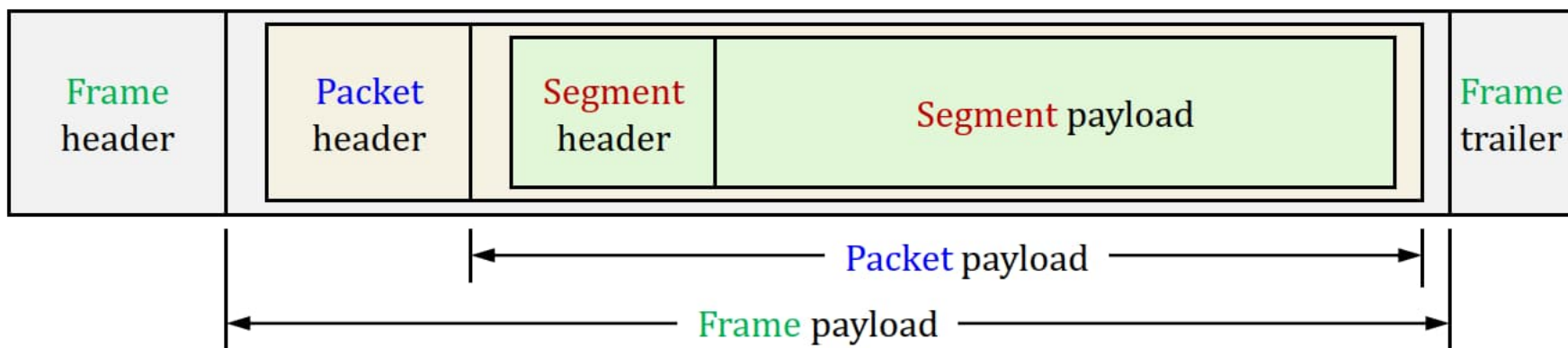


Figure 6-3. Nesting of segments, packets, and frames.



# State Diagram

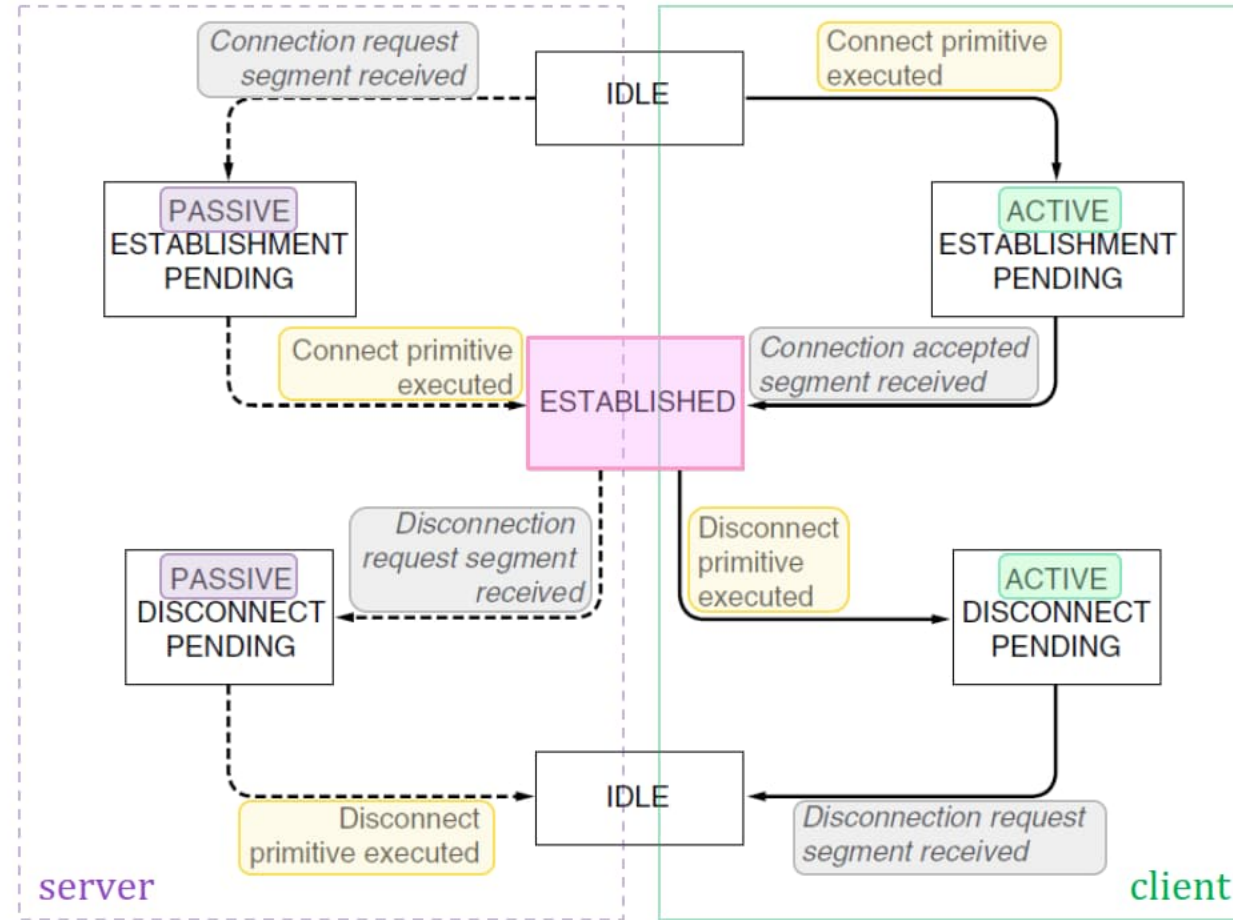


Figure 6-4. A state diagram for a simple connection management scheme. Transitions labeled in *italics* are caused by packet arrivals.



## 6.1.3 Berkeley Sockets



| Primitive | Meaning                                                     | end point           |
|-----------|-------------------------------------------------------------|---------------------|
| SOCKET    | Create a <b>new</b> communication <b>end point</b>          | <b>C</b> , <b>S</b> |
| BIND      | Attach a <b>local address</b> to a socket                   | <b>S</b>            |
| LISTEN    | Announce willingness to accept connections; give queue size | <b>S</b>            |
| ACCEPT    | <b>Block</b> the caller until a connection attempt arrives  | <b>S</b>            |
| CONNECT   | Actively attempt to establish a connection                  | <b>C</b>            |
| SEND      | Send some data over the connection ( <b>write</b> )         | <b>C</b> , <b>S</b> |
| RECEIVE   | Receive some data from the connection ( <b>read</b> )       | <b>C</b> , <b>S</b> |
| CLOSE     | <b>Release</b> the connection                               | <b>C</b> , <b>S</b> |

Figure 6-5. The socket primitives for TCP.

## 6.1.4 An Example of Socket Programming: An Internet File Server

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 12345 /* arbitrary, but client and server must agree */
#define BUF_SIZE 4096 /* block transfer size */

fatal(char *string)
{
    printf("%s\n", string);
    exit(1);
}

int main(int argc, char **argv)
{
    int c, s, bytes;
    char buf[BUF_SIZE]; /* buffer for incoming file */
    struct hostent *h; /* info about server */
    struct sockaddr_in channel; /* holds IP address */

    if (argc != 3) fatal("Usage: client server-name file-name");
```

Client code 1/2

Figure 6-6. Client code using sockets.

# Client code using sockets-2

Client code 2/2

```
h = gethostbyname(argv[1]);    /* look up host's IP address */
if (!h) fatal("gethostbyname failed");

s = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
if (s < 0) fatal("socket");
memset(&channel, 0, sizeof(channel));
channel.sin_family= AF_INET;
memcpy(&channel.sin_addr.s_addr, h->h_addr, h->h_length);
channel.sin_port= htons(SERVER_PORT);
c = connect(s, (struct sockaddr *) &channel, sizeof(channel));
if (c < 0) fatal("connect failed");

/* Connection is now established. Send file name including 0 byte at end. */
write(s, argv[2], strlen(argv[2])+1);

/* Go get the file and write it to standard output. */
while (1) {
    bytes = read(s, buf, BUF_SIZE);    /* read from socket */
    if (bytes <= 0) exit(0);           /* check for end of file */
    write(1, buf, bytes);              /* write to standard output */
}
}
```

Figure 6-6. Client code using sockets.



# Server code using sockets-1



Server code 1/2

```
#include <sys/types.h>
#include <sys/fcntl.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 12345 /* arbitrary, but client and server must agree */
#define BUF_SIZE 4096 /* block transfer size */
#define QUEUE_SIZE 10

int main(int argc, char *argv[])
{
    int s, b, l, fd, sa, bytes, on = 1;
    char buf[BUF_SIZE]; /* buffer for outgoing file */
    struct sockaddr_in channel; /* hold's IP address */

    /* Build address structure to bind to socket. */
    memset(&channel, 0, sizeof(channel)); /* zero channel */
    channel.sin_family = AF_INET;
    channel.sin_addr.s_addr = htonl(INADDR_ANY);
    channel.sin_port = htons(SERVER_PORT);

    /* Passive open. Wait for connection. */
    s = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP); /* create socket */
    if (s < 0) fatal("socket failed");
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, (char *) &on, sizeof(on));
```

# Server code using sockets-2

Server code 2/2

```
b = bind(s, (struct sockaddr *) &channel, sizeof(channel));
if (b < 0) fatal("bind failed");

l = listen(s, QUEUE_SIZE);           /* specify queue size */
if (l < 0) fatal("listen failed");

/* Socket is now set up and bound. Wait for connection and process it. */
while (1) {
    sa = accept(s, 0, 0);             /* block for connection request */
    if (sa < 0) fatal("accept failed");

    read(sa, buf, BUF_SIZE);          /* read file name from socket */

    /* Get and return the file. */
    fd = open(buf, O_RDONLY);          /* open the file to be sent back */
    if (fd < 0) fatal("open failed");

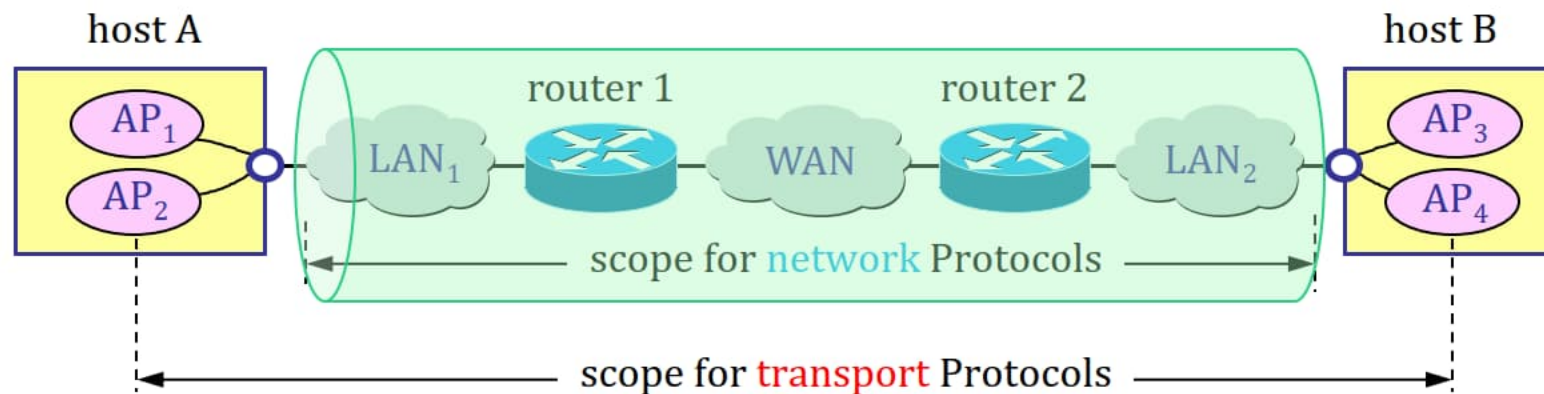
    while (1) {
        bytes = read(fd, buf, BUF_SIZE); /* read from file */
        if (bytes <= 0) break;           /* check for end of file */
        write(sa, buf, bytes);           /* write bytes to socket */
    }
    close(fd);                         /* close file */
    close(sa);                         /* close connection */
}
}
```



## 6.2 Elements of Transport Protocols

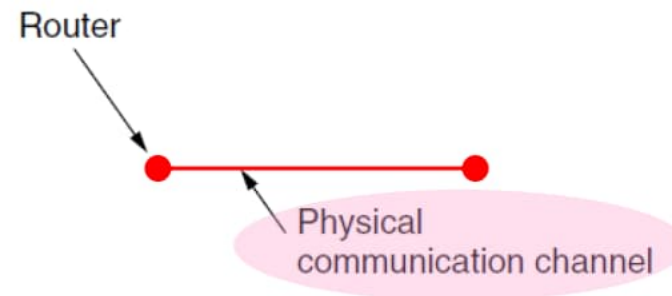
### ➤ Elements of Transport Protocols

- ✦ Addressing 编址
- ✦ Connection Establishment 建立连接
- ✦ Connection Release 释放连接
- ✦ Error Control and Flow Control 差错控制 与 流量控制
- ✦ Multiplexing 复用
- ✦ Crash Recovery 崩溃后恢复

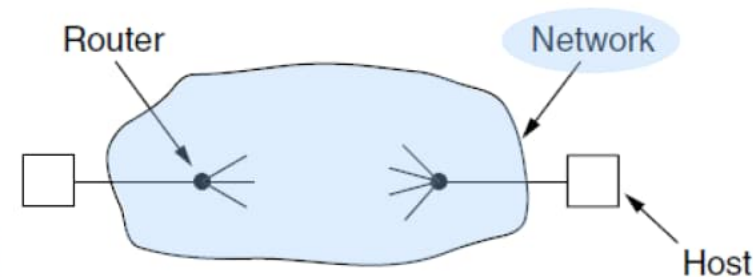


# the data link layer and the transport layer

- Comparison between data link layer and transport layer
  - ✦ similarity
    - error control, sequencing, and flow control
  - ✦ differences
    - directly via a physical channel vs. entire network



(a) Environment of the data link layer.



(b) Environment of the transport layer.

Figure 6-7.

# 6.2.1 Addressing

## ➤ Addressing 编址

- ★ **TSAP**: Transport Service Access Point 传输层服务访问点
- ★ **NSAP**: Network Service Access Point 网络层服务访问点

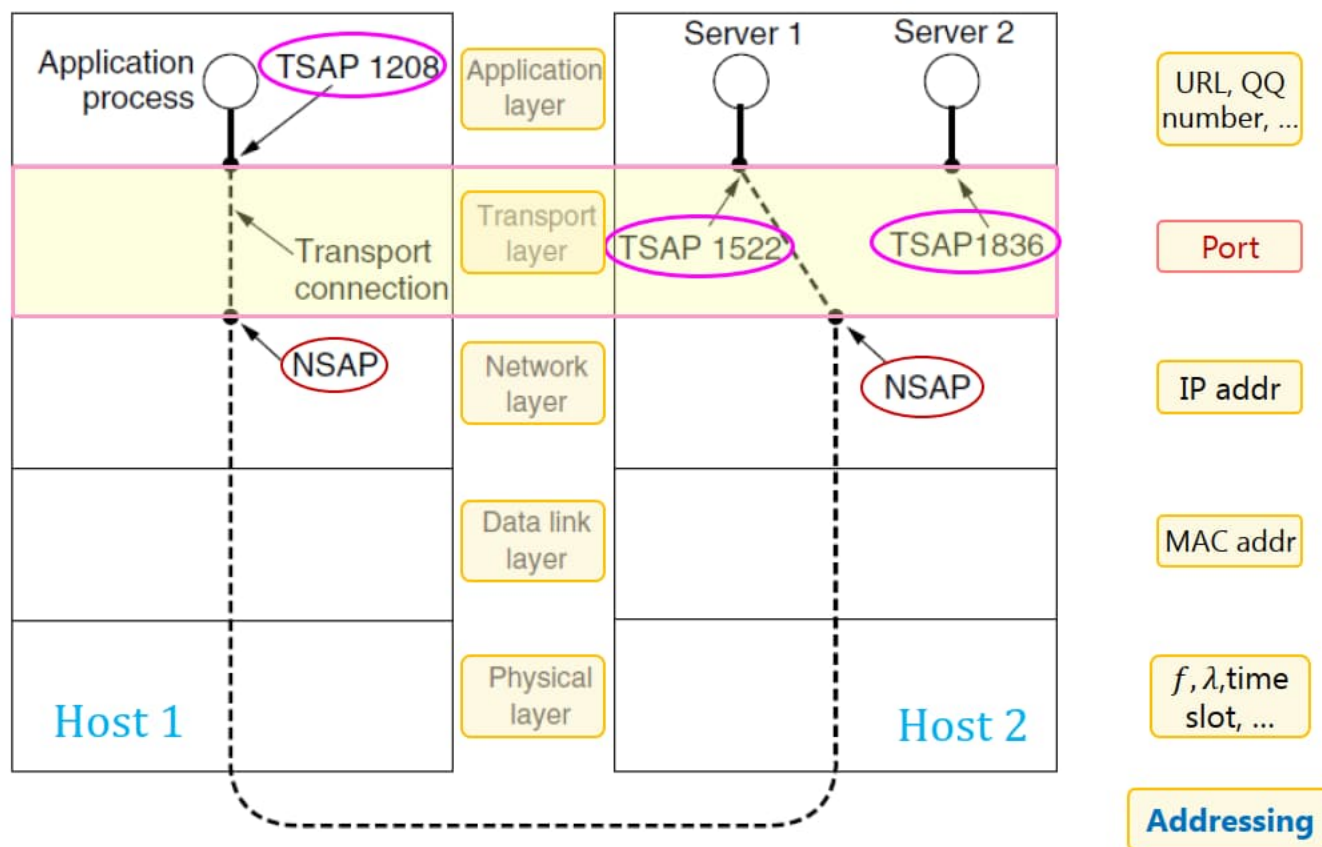


Figure 6-8. TSAPs, NSAPs, and transport connections.

# initial connection protocol

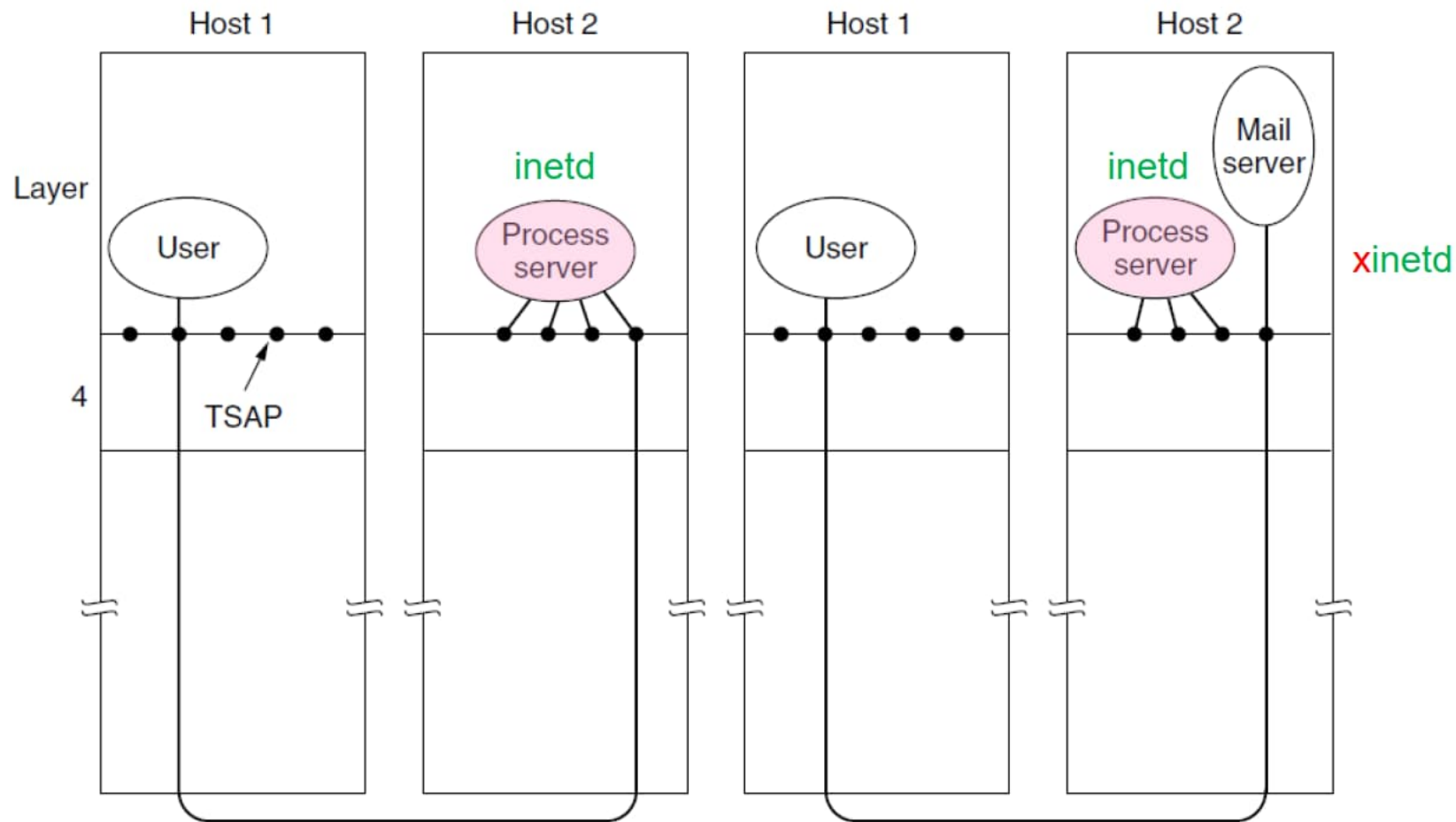


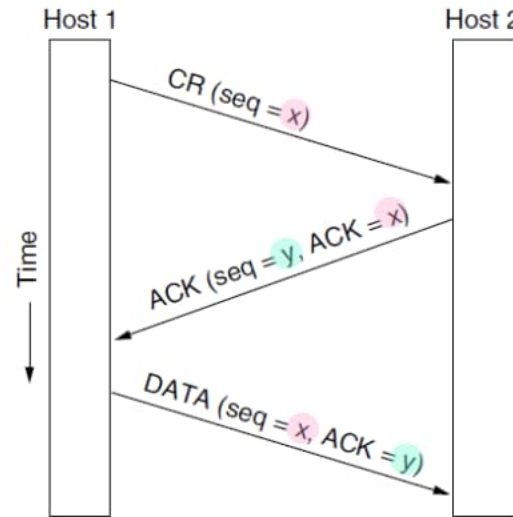
Figure 6-9. How a user process in host 1 establishes a connection with a mail server in host 2 via a **process server**.



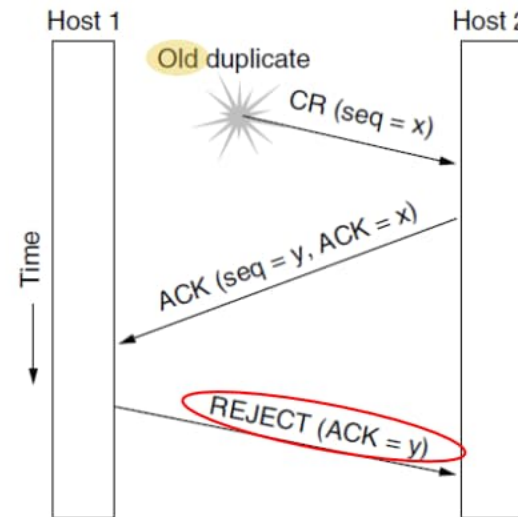
## 6.2.2 Connection Establishment

### ➤ three-way handshake

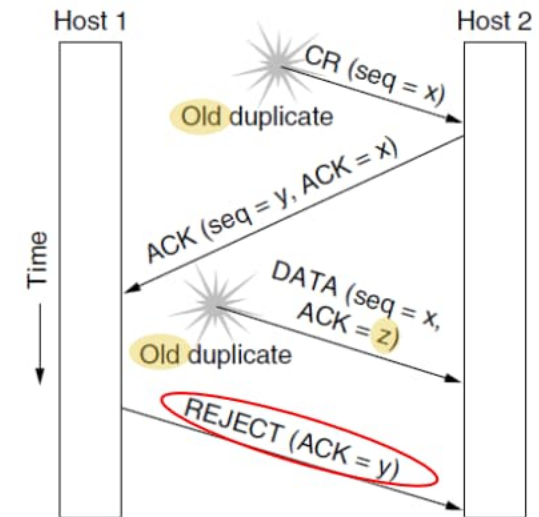
三路握手



(a) Normal operation.



(b) Old duplicate CONNECTION REQUEST appearing out of nowhere.



(c) Duplicate CONNECTION REQUEST and duplicate ACK.

Figure 6-11. Three protocol scenarios for establishing a connection using a three-way handshake. CR denotes CONNECTION REQUEST.

## 6.2.3 Connection Release

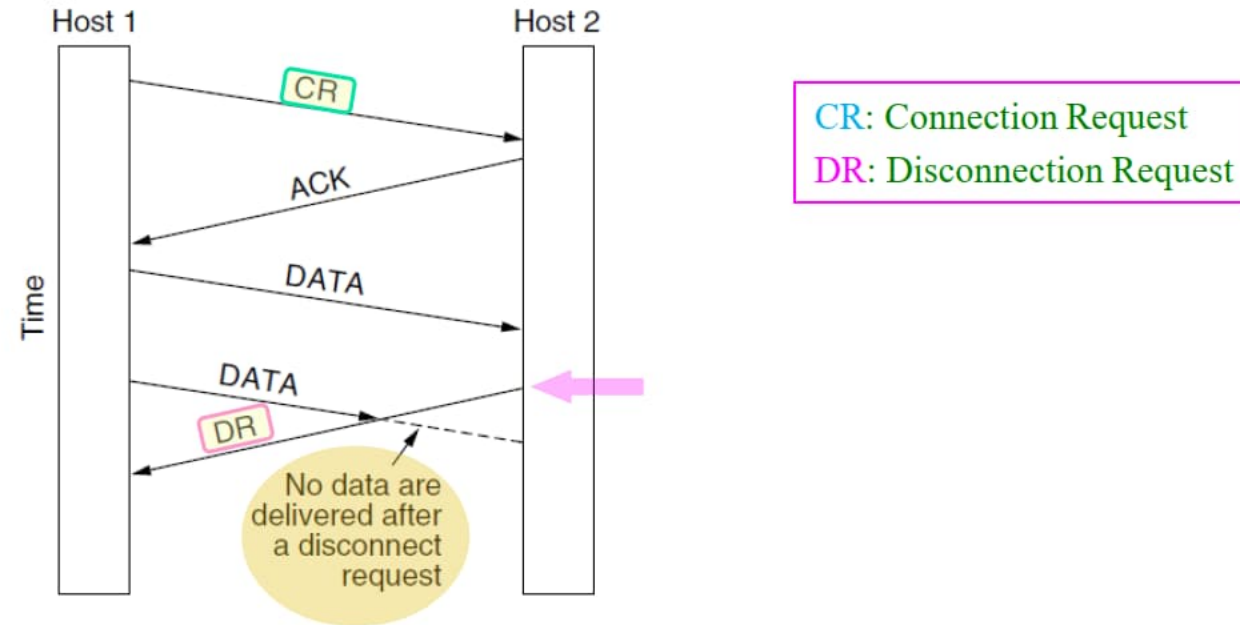


Figure 6-12. Abrupt disconnection with loss of data.

# The two-army problem

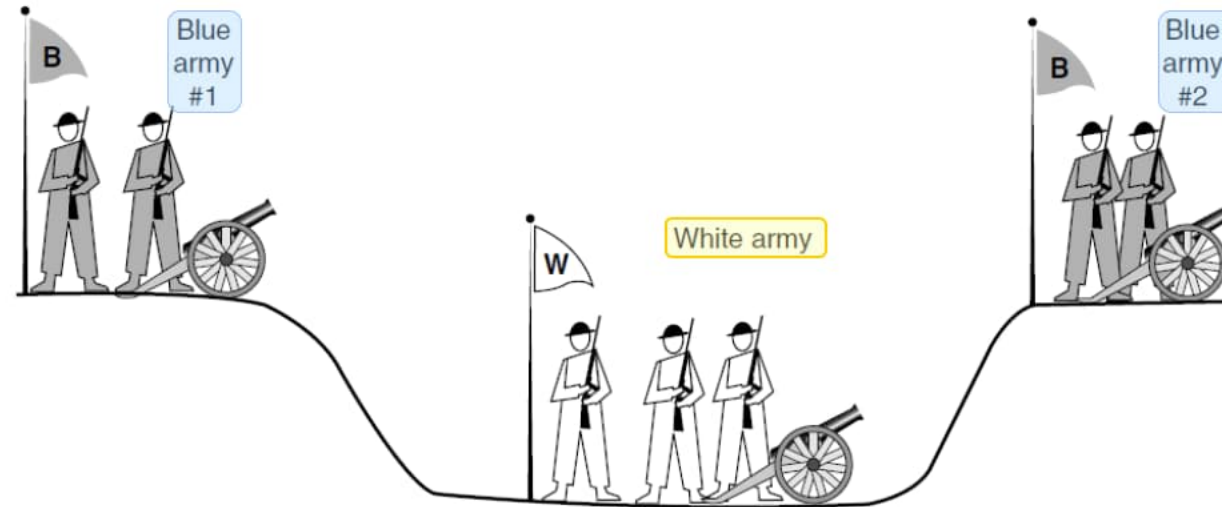
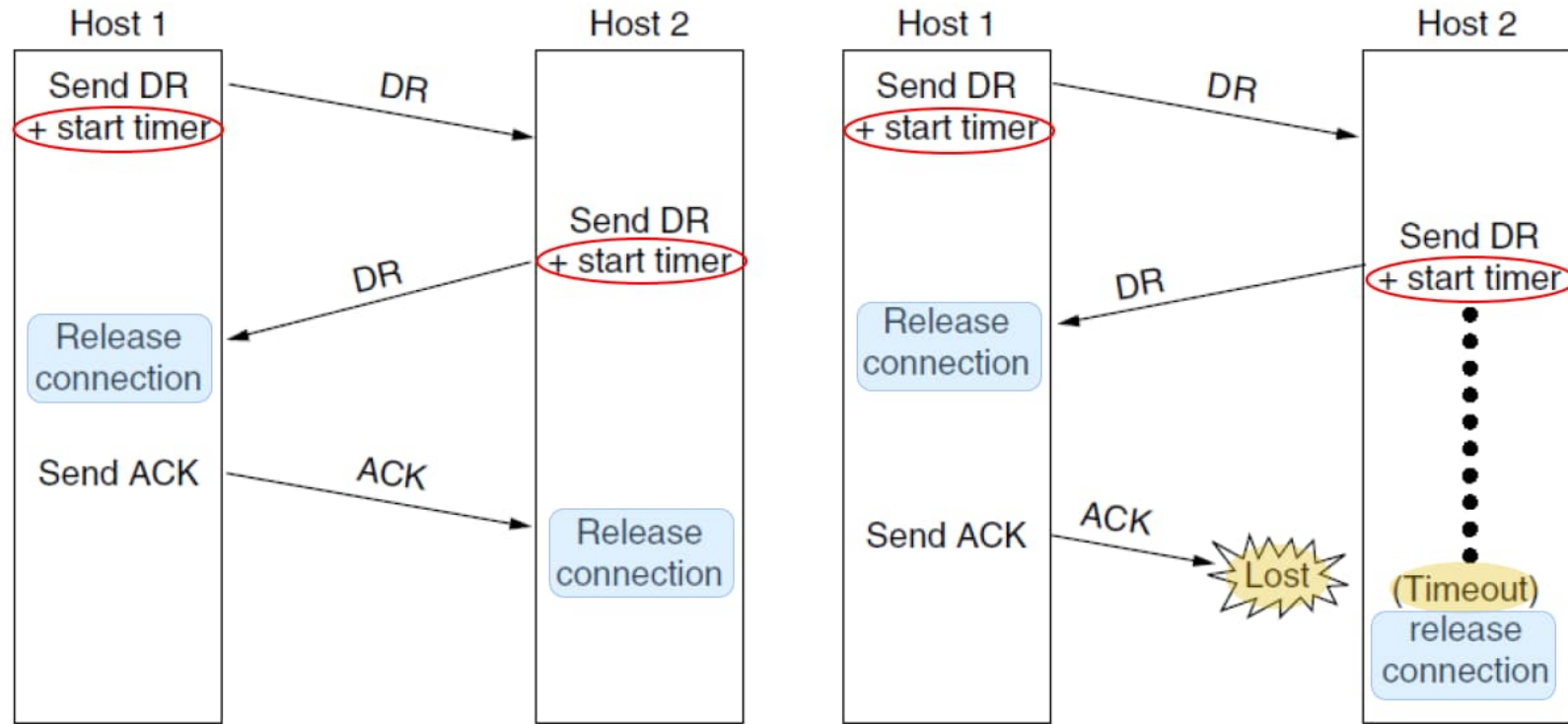


Figure 6-13. The two-army problem.

# connection release -1



(a) Normal case of three-way handshake.

(b) Final ACK lost

Figure 6-14. Four protocol scenarios for releasing a connection.





(d) Response lost and subsequent DRs lost.

Figure 6-14. Four protocol scenarios for releasing a connection.

## 6.2.4 Error Control and Flow Control

### ➤ Error control

- ✦ is ensuring that the data is delivered with the desired level of **reliability**, usually that all of the data is delivered **without any errors**.

### ➤ Flow control

- ✦ is **keeping** a fast transmitter **from overrunning** a slow receiver.

# Solutions recap at the Link Layer

## 链路层方案回顾

- The **solutions** that are used at the **transport layer** are **the same** mechanisms that we studied at the **link layer in Chap. 3**.
  1. A frame carries an **error-detecting code** (e.g., a CRC or checksum) that is used to check if the information was correctly received. 检错码
  2. A frame carries a **sequence number** to identify itself and is retransmitted by the sender until it receives an **acknowledgement** of successful receipt from the receiver. 顺序号
    - This is called **ARQ** (**A**utomatic **R**epeat re**Q**uest).
  3. There is a **maximum number of frames** that the sender will allow to be outstanding at any time, pausing if the receiver is not acknowledging frames quickly enough. 同时发送
    - If this maximum is **one** packet the protocol is called **stop-and-wait**.
    - **Larger windows** enable pipelining and **improve performance**.
  4. The **sliding window protocol** combines these features and is also used to support bidirectional data transfer. 滑动窗口

recap [ˌrɪˈkæp] *n.* 扼要重述; 概括; 回顾

outstanding [aʊtˈstændɪŋ] *adj.* 杰出的; 未完成的, 未解决的



# Differences between Link layer and Transport layer

## ➤ differences in function 功能上

- ✦ The **link layer checksum** protects a frame while it crosses **a single link**.
- ✦ The **transport layer checksum** protects a segment while it crosses **an entire network path**. It is an end-to-end check.

## ➤ difference in degree 程度上

- ✦ the **bandwidth-delay product** for **the link** is **small enough** that not even a whole frame can be stored inside the link. BDP(带宽时延积)极小
  - a **small window size** is **sufficient** for good performance.
- ✦ **TCP connections** have a **bandwidth-delay product** that is **much larger** than a single segment. BDP(带宽时延积)很大
  - a **large sliding window** must be used.
  - Stop-and-wait will cripple performance.

BDP: Bandwidth-Delay Product, 带宽时延积  
cripple ['krip(ə)l] *n.* 跛子; *v.* 使残废; 损害



# buffering data

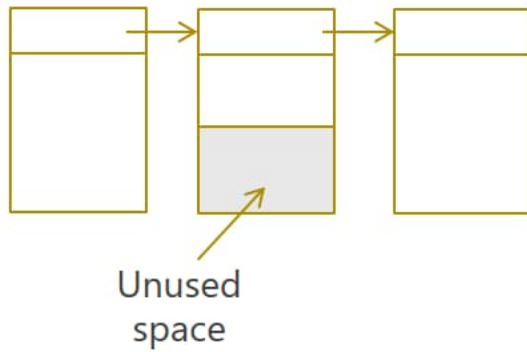


- The **buffers** are needed at **both** the **sender** and the **receiver**.
  - ✦ buffers are needed **at the sender** to hold all transmitted but as yet **unacknowledged** segments. They are needed there because these segments **may be lost** and need to be **retransmitted**. 发送方缓存
  - ✦ **The receiver may or may not dedicate** specific buffers to specific connections. The receiver may maintain **a single buffer pool shared by all connections**. 接收方缓存（共享）
- The best **trade-off** between **source buffering** and **destination buffering**
  - ✦ depends on the type of **traffic** carried by the connection.
    - For **low-bandwidth bursty** traffic, it is **not** to **dedicate** any buffers. 低速
      - such as that produced by an **interactive** terminal
    - for file transfer and other **high-bandwidth** traffic, it is better if the **receiver does dedicate** a full window of buffers, to allow the data to flow at maximum speed. 高速
      - This is the strategy that **TCP** uses.

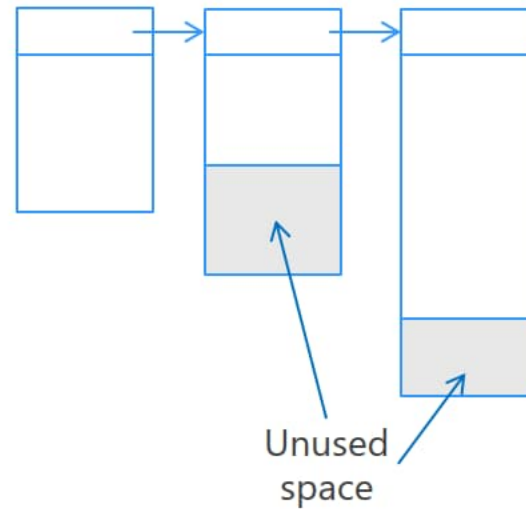
# the buffer pool organization



(a) Chained **fixed-size** buffers.



(b) Chained **variable-sized** buffers.



(c) One large **circular** buffer per connection.

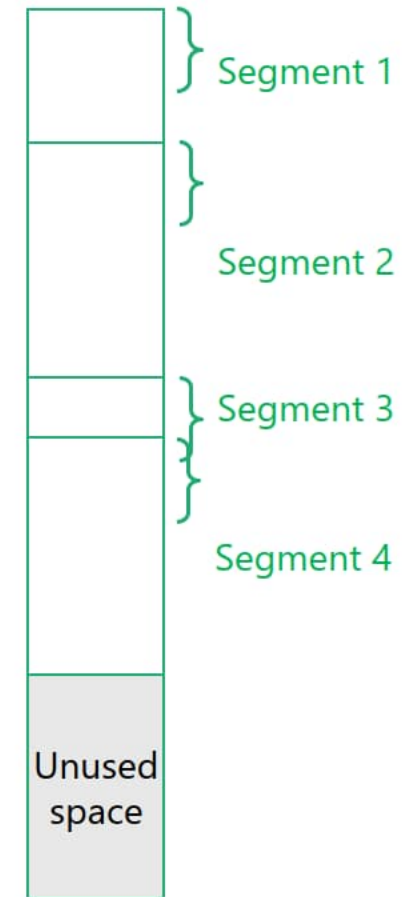


Figure 6-15.

# dynamic window management



| No. | A   | Message              | B   | Comments                                     |
|-----|-----|----------------------|-----|----------------------------------------------|
| 1   | →   | <request 8 buffers>  | →   | A wants 8 buffers                            |
| 2   | ←   | <ack = 15, buf = 4>  | ←   | B grants messages 0-3 only                   |
| 3   | →   | <seq = 0, data = m0> | →   | A has 3 buffers left now                     |
| 4   | →   | <seq = 1, data = m1> | →   | A has 2 buffers left now                     |
| 5   | →   | <seq = 2, data = m2> | ... | Message lost but A thinks it has 1 left      |
| 6   | ←   | <ack = 1, buf = 3>   | ←   | B acknowledges 0 and 1, permits 2-4          |
| 7   | →   | <seq = 3, data = m3> | →   | A has 1 buffer left                          |
| 8   | →   | <seq = 4, data = m4> | →   | A has 0 buffers left, and must stop          |
| 9   | →   | <seq = 2, data = m2> | →   | A times out and retransmits                  |
| 10  | ←   | <ack = 4, buf = 0>   | ←   | Everything acknowledged, but A still blocked |
| 11  | ←   | <ack = 4, buf = 1>   | ←   | A may now send 5                             |
| 12  | ←   | <ack = 4, buf = 2>   | ←   | B found a new buffer somewhere               |
| 13  | →   | <seq = 5, data = m5> | →   | A has 1 buffer left                          |
| 14  | →   | <seq = 6, data = m6> | →   | A is now blocked again                       |
| 15  | ←   | <ack = 6, buf = 0>   | ←   | A is still blocked                           |
| 16  | ... | <ack = 6, buf = 4>   | ←   | Potential deadlock                           |

Figure 6-16. Dynamic buffer allocation. The arrows show the direction of transmission. An ellipsis (...) indicates a lost segment.

## 6.2.5 Multiplexing

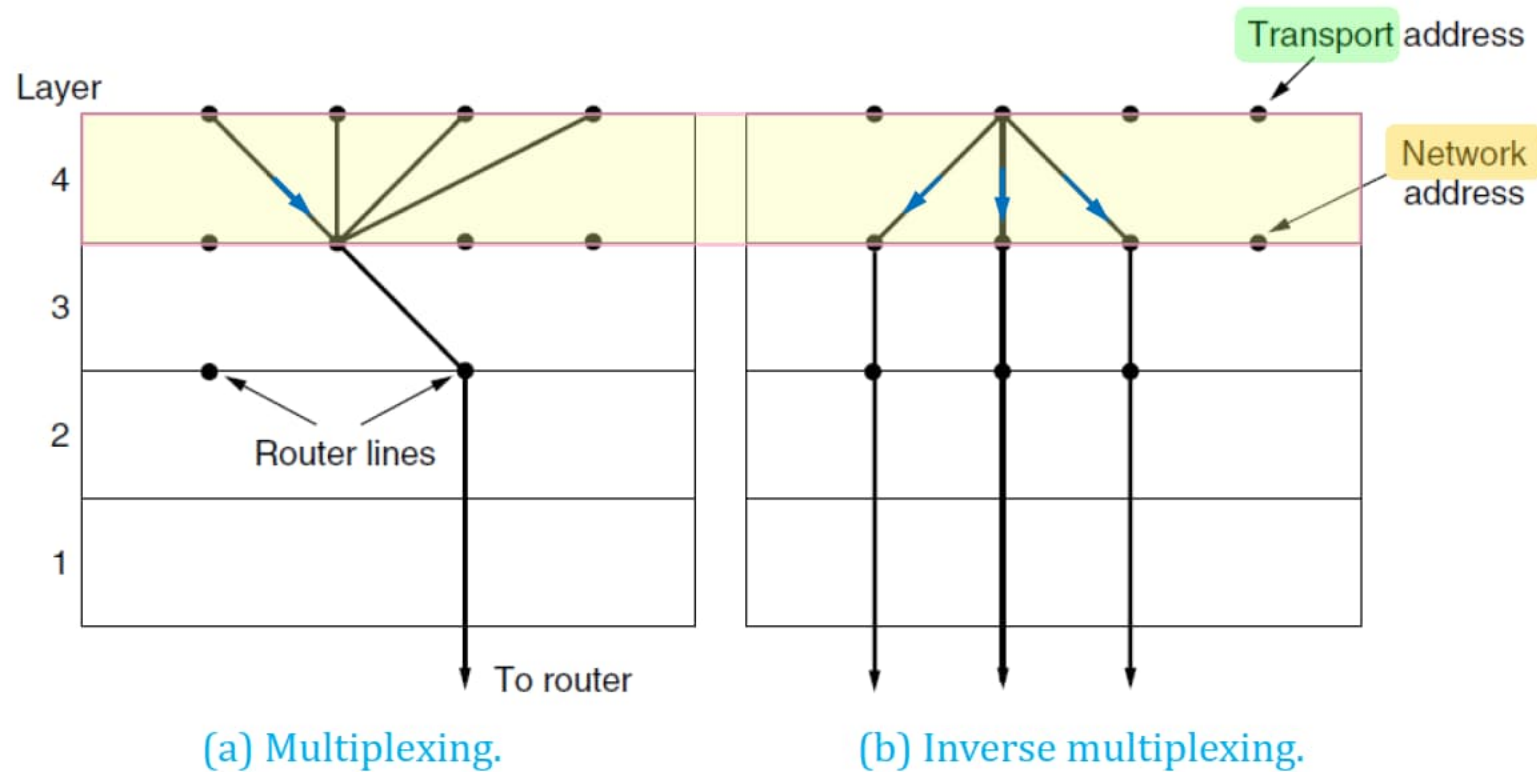


Figure 6-17.



## 6.2.6 Crash Recovery

- ✦ No matter how the client and server are programmed, there are always situations where the protocol fails to recover properly.
- ✦ Recovery from a layer N crash can only be done by layer N + 1
  - only if the higher layer retains enough status information

|                       |                   | Strategy used by receiving host |     |       |                       |     |       |
|-----------------------|-------------------|---------------------------------|-----|-------|-----------------------|-----|-------|
|                       |                   | First ACK, then write           |     |       | First write, then ACK |     |       |
|                       |                   | Strategy used by Sending host   |     |       |                       |     |       |
|                       |                   | AC(W)                           | AWC | C(AW) | C(WA)                 | WAC | WC(A) |
| Server → Client crash | Always retransmit | OK                              | DUP | OK    | OK                    | DUP | DUP   |
|                       | Never retransmit  | LOST                            | OK  | LOST  | LOST                  | OK  | OK    |
| Client → Server crash | Retransmit in S0  | OK                              | DUP | LOST  | LOST                  | DUP | OK    |
|                       | Retransmit in S1  | LOST                            | OK  | OK    | OK                    | OK  | DUP   |

S0: no segments outstanding

S1: one segment outstanding

OK = Protocol functions correctly

DUP = Protocol generates a duplicate message

LOST = Protocol loses a message

A: Ack

W: Write

C: Crash

Figure 6-18. Different combinations of client and server strategies.

outstanding *adj.* 未完成的; 未解决的;

## 6.3 Congestion Control

- Congestion
  - ✦ If the transport entities on **many machines**
    - send **too many packets** into the network **too quickly**,
    - the network will become **congested**,
    - with **performance degraded** as packets are **delayed** and **lost**.
- Congestion Control mechanisms
  - ✦ in the **network** layer
  - ✦ in the **transport** layer
- Subjects
  - ✦ Desirable Bandwidth Allocation 期望的带宽分配
  - ✦ Regulating the Sending Rate 发送速率调节
  - ✦ Wireless Issues 无线问题

## 6.3.1 Desirable Bandwidth Allocation

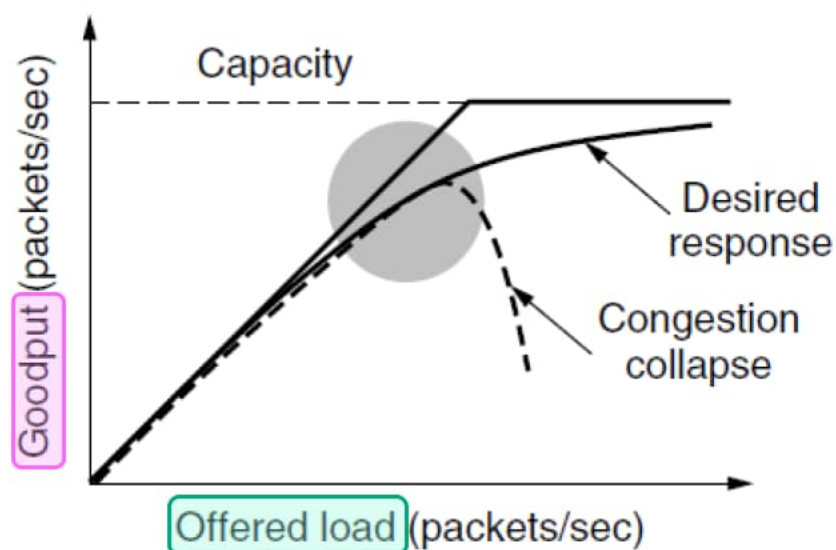
- The goals of congestion control
  - ✦ avoid congestion
  - ✦ use all the available band-width
  - ✦ be **fair** across competing transport entities
  - ✦ quickly track changes in traffic demands.
- Subjects of bandwidth allocation
  - ✦ Efficiency and Power 效率与功率
  - ✦ Max-Min Fairness 最大最小公平算法
  - ✦ Convergence 收敛性

# Efficiency and Power

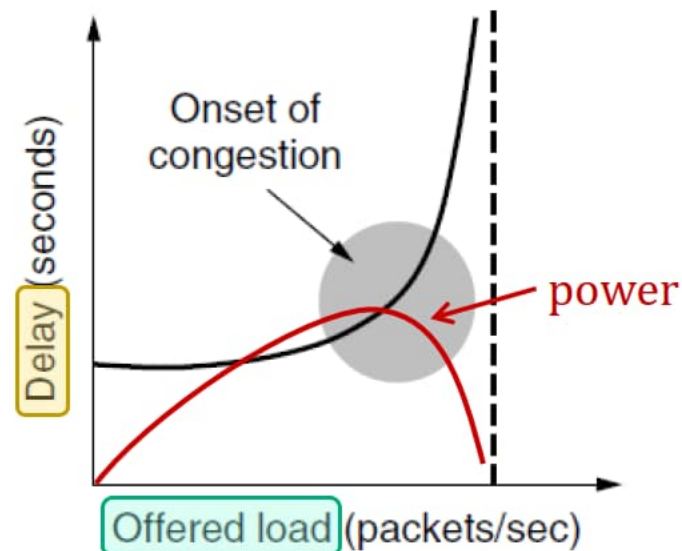


## ➤ Efficiency and Power 效率与功率

$$* \text{ power} = \frac{\text{load}}{\text{delay}}$$



(a) Goodput as a function of offered load.



(b) Delay as a function of offered load.

Figure 6-19.



# Max-Min Fairness

## ➤ Max-Min Fairness 最大-最小公平算法

★ An allocation is max-min fair

- if the **bandwidth** given to **one flow** **cannot be increased**
- **without decreasing** the bandwidth given to **another flow** with an allocation
- that is **no larger**.

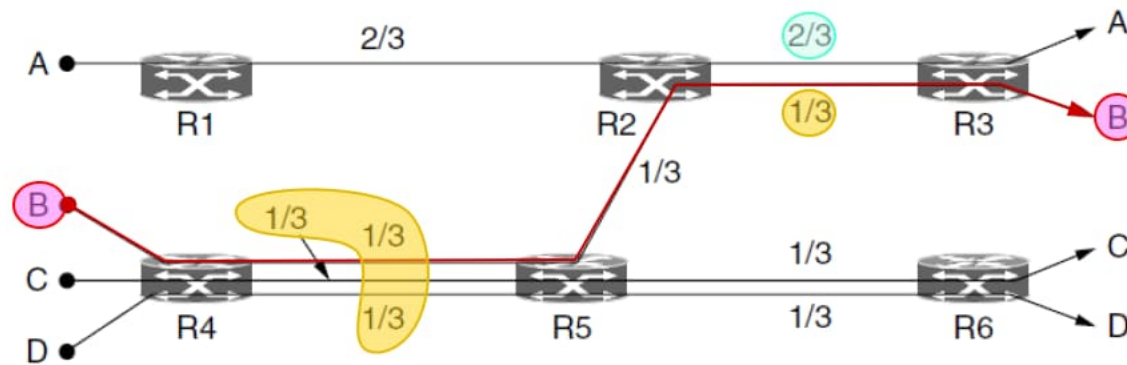


Figure 6-20. Max-min bandwidth allocation for four flows.

| Max-Min Fairness |       |       |      | (20) |
|------------------|-------|-------|------|------|
| 1,               | 6,    | 8,    | 9    |      |
| =====            |       |       |      |      |
| 5,               | 5,    | 5,    | 5    |      |
| 1,               | 6.33, | 6.33, | 6.33 |      |
| 1,               | 6,    | 6.5,  | 6.5  |      |

| Max-Min Fairness |    |      |     | (20) |
|------------------|----|------|-----|------|
| 1,               | 6, | 8,   | 9   |      |
| =====            |    |      |     |      |
| 1,               | 1, | 1,   | 1   | (16) |
| 1,               | 6, | 6,   | 6   | (1)  |
| 1,               | 6, | 6.5, | 6.5 | (0)  |

# Convergence

- A **good** congestion control algorithm
  - ✦ should **rapidly converge** to the **ideal** operating point.

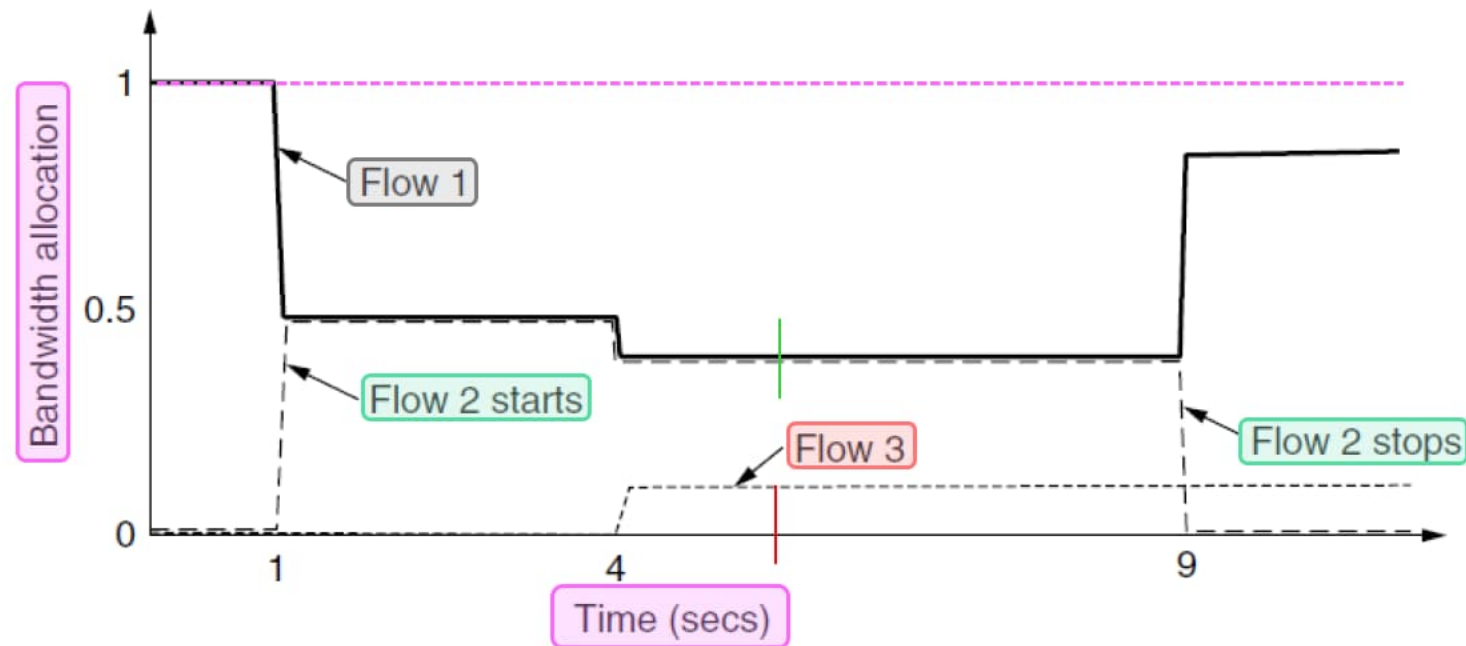


Figure 6-21. Changing bandwidth allocation over time.

## 6.3.2 Regulating the Sending Rate

### ➤ flow control vs. congestion control

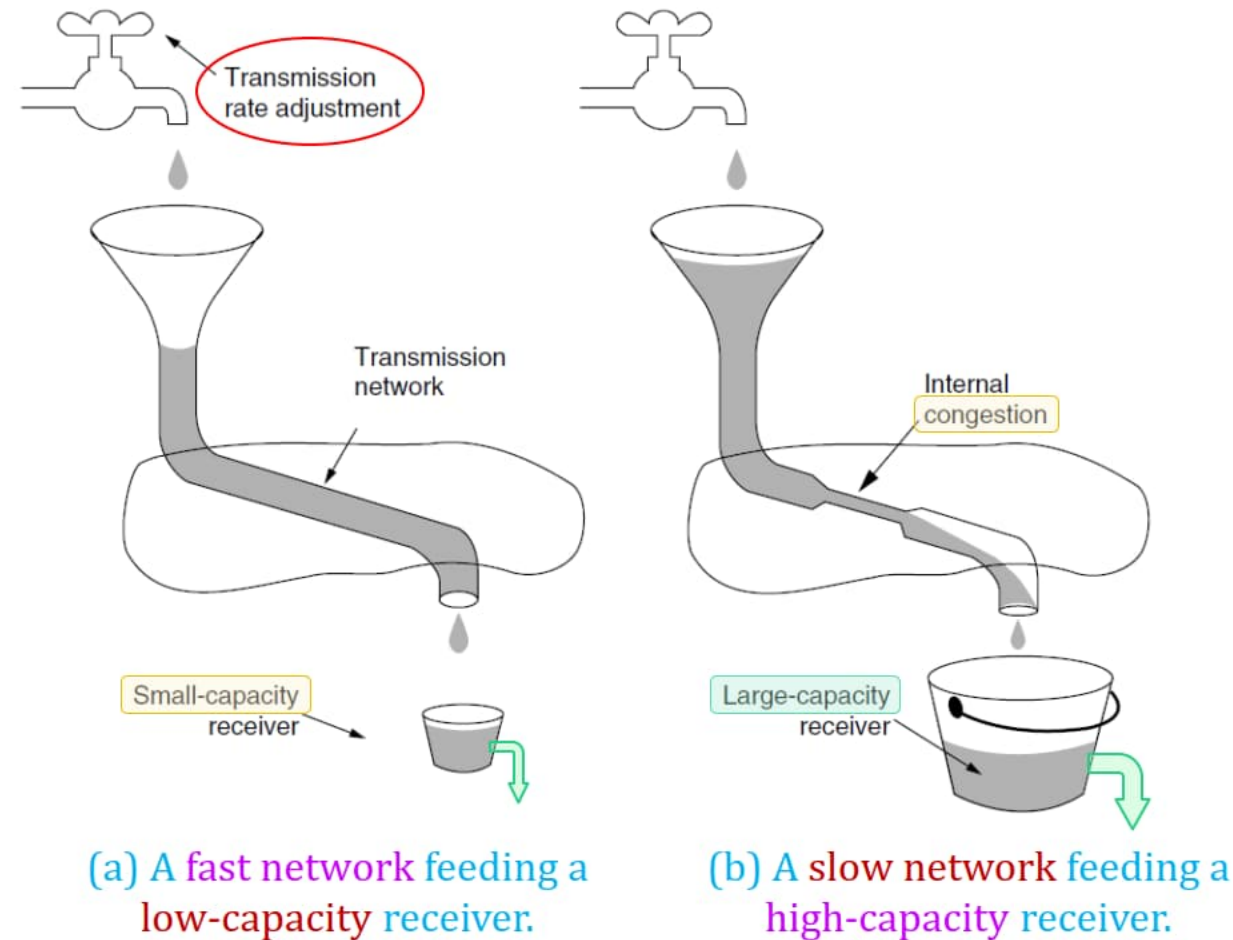


Figure 6-22.

# the form of the feedback



## ➤ The feedback

- ✦ may be **explicit** or **implicit**      显式 or 隐式
- ✦ may be **precise** or **imprecise**.      精确 or 模糊

| Protocol     | Signal                         | Explicit | Precise |
|--------------|--------------------------------|----------|---------|
| XCP          | Rate to use                    | Yes      | Yes     |
| TCP with ECN | Congestion warning             | Yes      | No      |
| FAST TCP     | End-to-end delay               | No       | Yes     |
| Compound TCP | Packet loss & end-to-end delay | No       | Yes     |
| CUBIC TCP    | Packet loss                    | No       | No      |
| TCP          | Packet loss                    | No       | No      |

Figure 6-23. Signals of some congestion control protocols.



## ➤ AIMD: Additive Increase Multiplicative Decrease

✦ AIMD 加性递增, 乘性递减

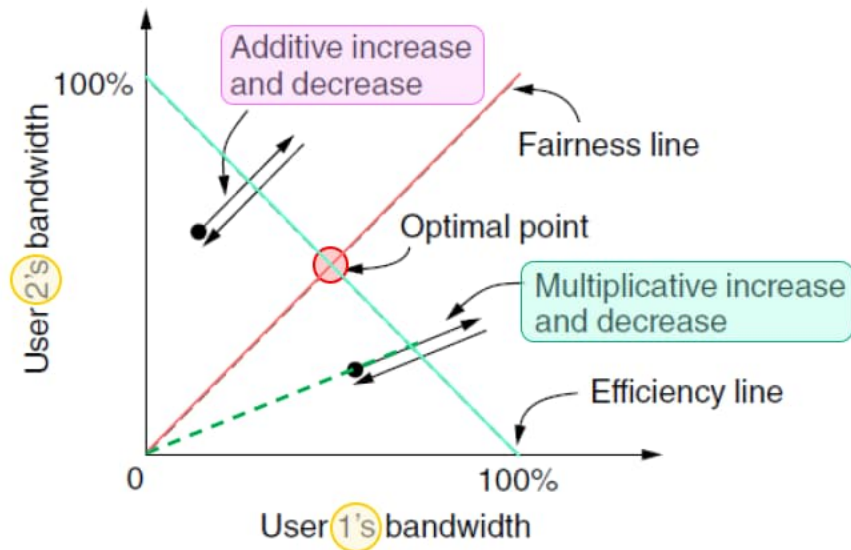


Figure 6-24. Additive and multiplicative bandwidth adjustments.

Additive increase (up at 45°)  
Multiplicative decrease (line points to origin)

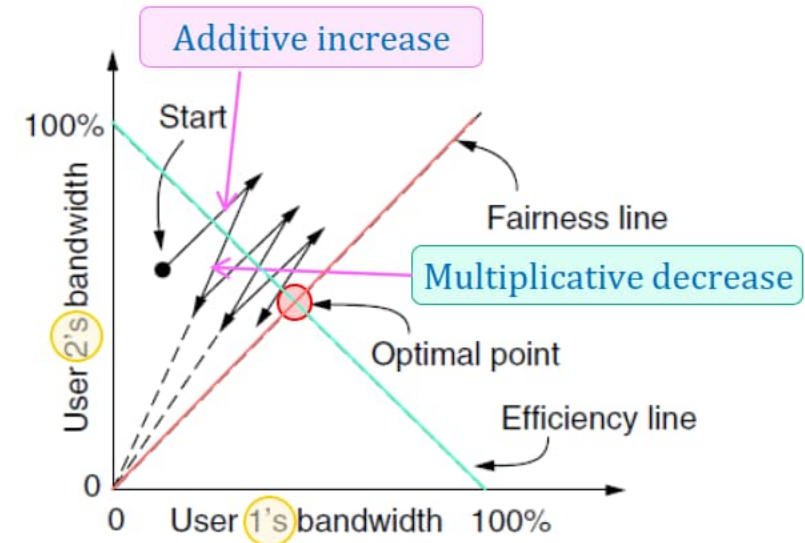


Figure 6-25. Additive Increase Multiplicative Decrease (AIMD) control law.

## 6.3.3 Wireless Issues

### ➤ Problem

- ✦ Packet loss is often used as a congestion signal.
- ✦ Wireless networks lose packets all the time due to transmission errors.

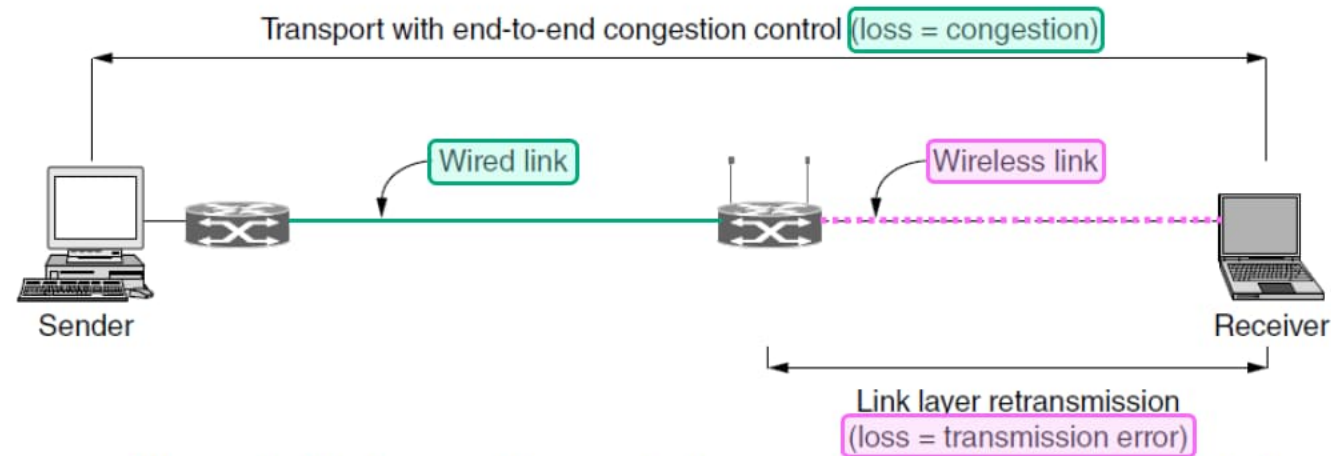


Figure 6-26. Congestion control over a path with a wireless link.

### ➤ Solution

- ✦ to mask the wireless losses by using
  - retransmissions over the wireless link.
    - 802.11 uses a stop-and-wait protocol to deliver each frame.
  - FEC (Forward Error Correction) in wireless links have long round-trip times.

## 6.4 The Internet Transport Protocols: UDP

- UDP: User Datagram Protocol      用户数据报协议
  - ✦ Connectionless
  - ✦ It **does almost nothing** beyond sending packets between applications.
  - ✦ RFC 768
- Subjects
  - ✦ Introduction to UDP
  - ✦ **RPC**: Remote Procedure Call
  - ✦ **RTP**: Real-Time Transport Protocols

## 6.4.1 Introduction to UDP

- UDP (User Datagram Protocol)
  - ✦ “no frills,” “bare bones” Internet transport protocol
  - ✦ PDU: segments
- Why is there a UDP?
  - ✦ no connection establishment (which can add delay)
  - ✦ simple: no connection state at sender, receiver
  - ✦ small segment header
  - ✦ no congestion control: UDP can blast away as fast as desired
- often used for streaming multimedia apps
- other UDP uses:
  - ✦ RIP (Routing Information Protocol)
  - ✦ DNS (Domain Name System)
  - ✦ SNMP (Simple Network Management Protocol)

frill *n.* 荷叶边、褶边、(不实用的)装饰



# The UDP header

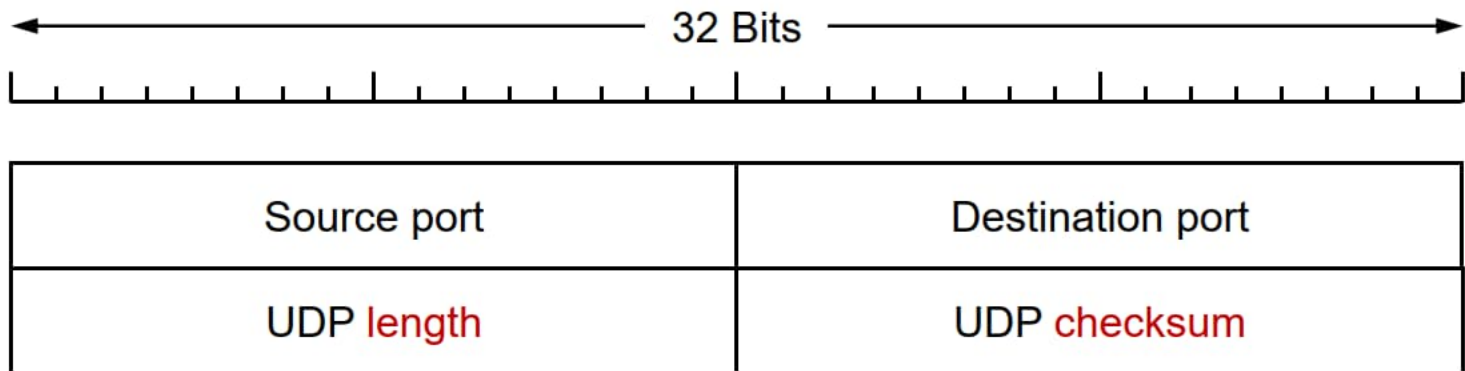


Figure 6-27. The UDP header.

- ✦ UDP length: total length, including the 8-byte header and the data.
- ✦ UDP Checksum: optional, checksums the UDP header, the data, and a conceptual IP pseudoheader.

# The IPv4 pseudoheader

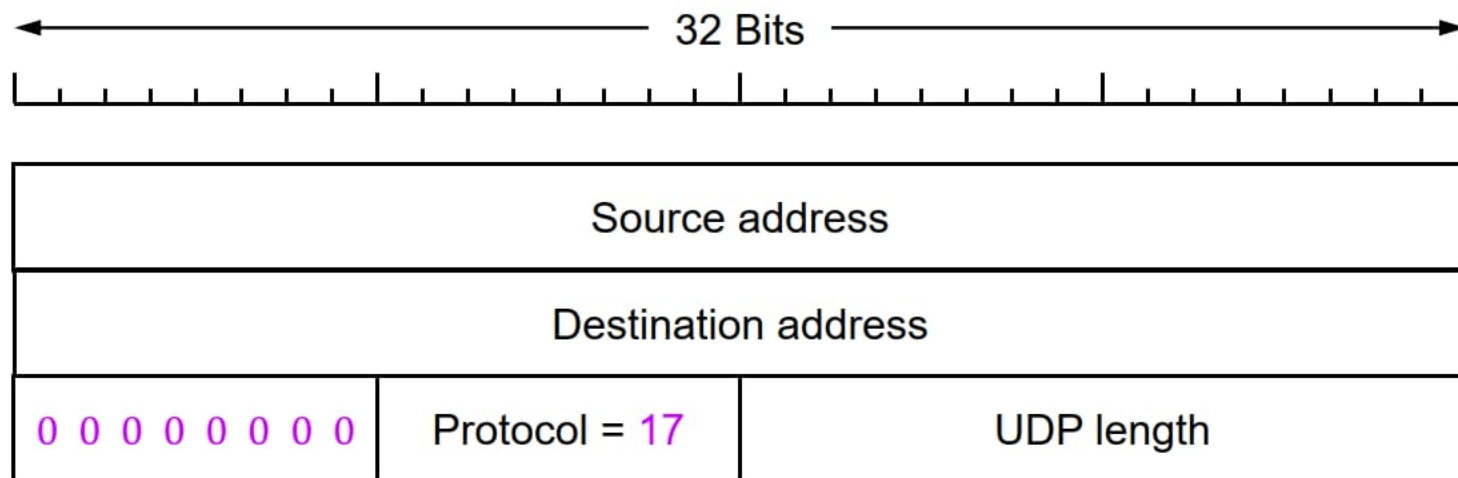
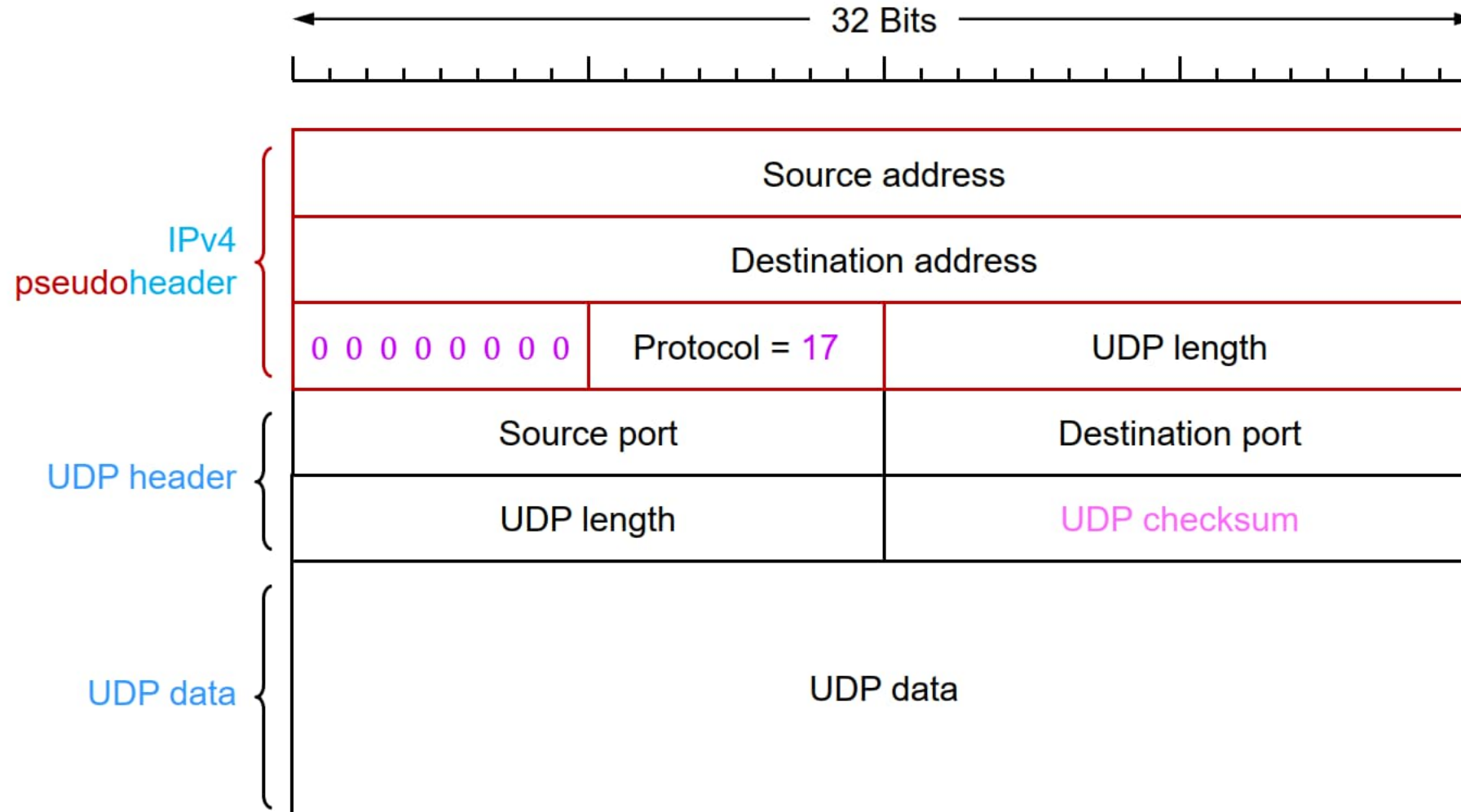


Figure 6-28. The IPv4 pseudoheader included in the UDP checksum.

# UDP checksum



## 6.4.2 Remote Procedure Call

- RPC: Remote Procedure Call 远程过程调用

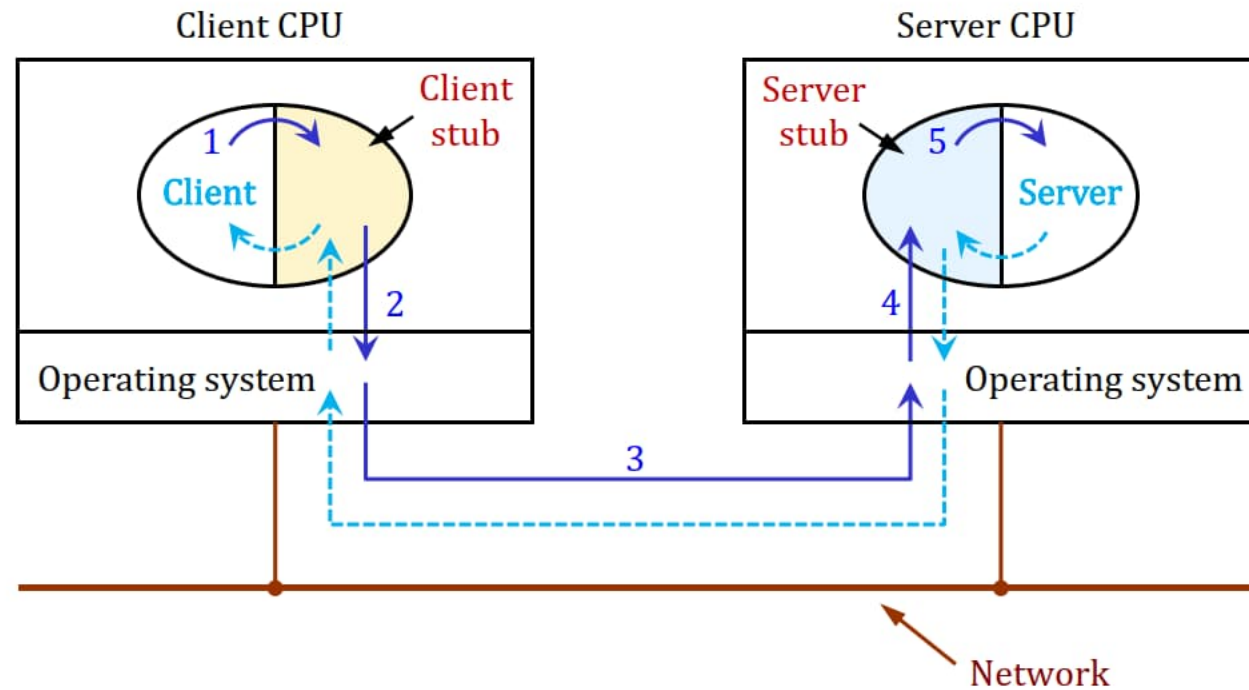


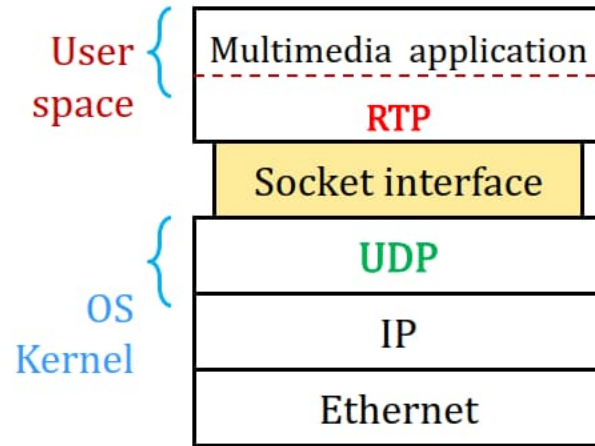
Figure 6-29. Steps in making a remote procedure call. The stubs are shaded.



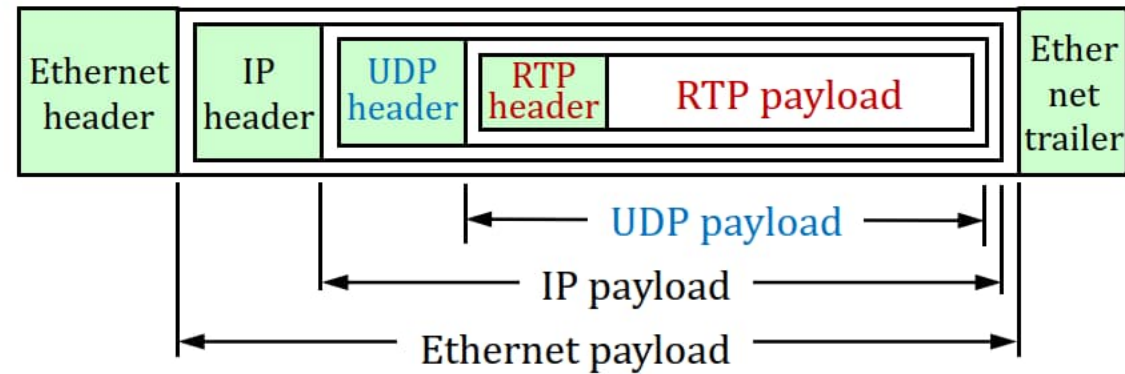
## 6.4.3 Real-Time Transport Protocols

### ➤ RTP: Real-time Transport Protocol

✦ RFC 3550



(a) The position of RTP in the protocol stack.



(b) Packet nesting.

Figure 6-30

# Playout with Buffering and Jitter Control

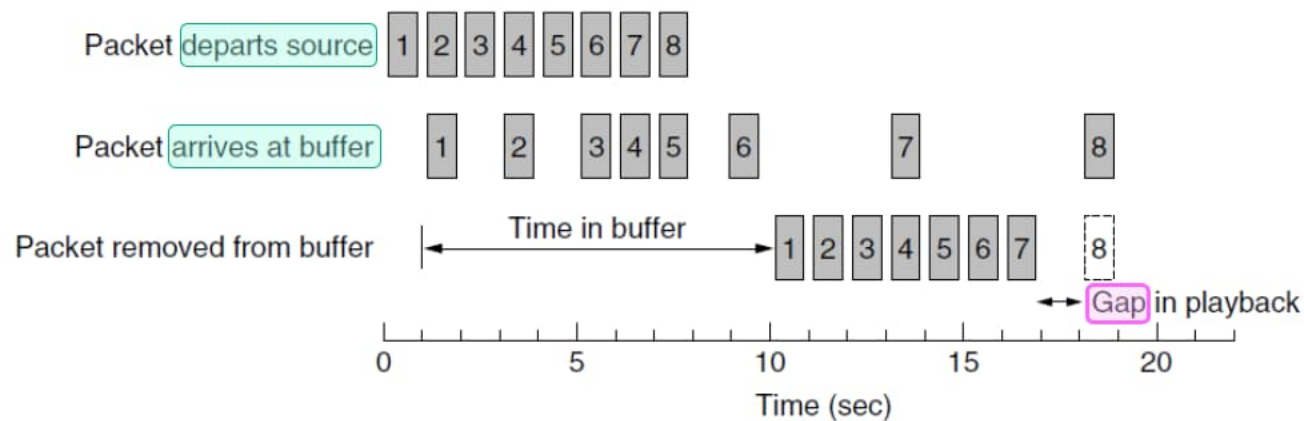


Figure 6-32. Smoothing the output stream by buffering packets.

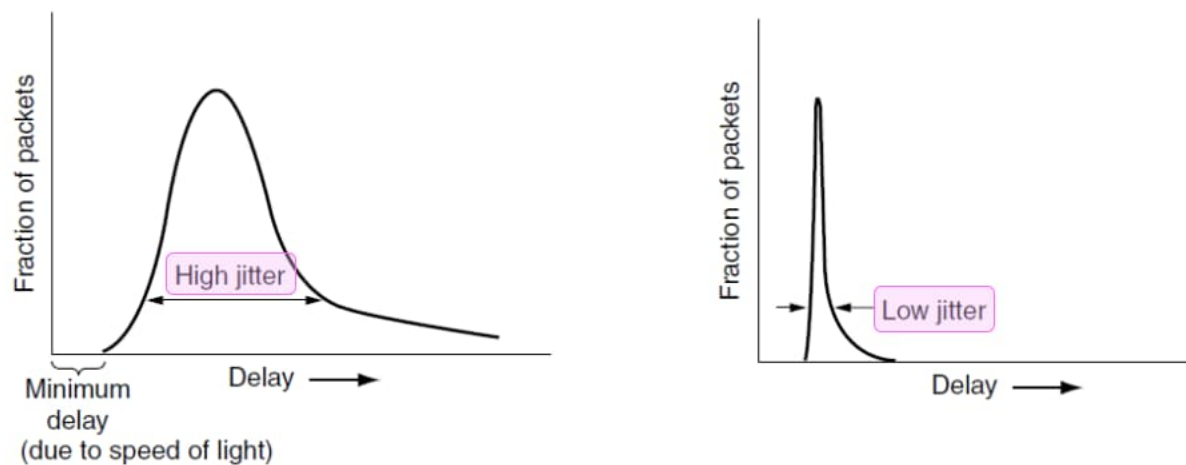


Figure 6-33. (a) High jitter.

Figure 6-33. (b) Low jitter.

## 6.5 The Internet Transport Protocols: TCP

- ✦ Introduction to TCP
- ✦ The TCP Service Model
- ✦ The TCP Protocol
- ✦ The TCP Segment Header
- ✦ TCP Connection Establishment
- ✦ TCP Connection Release
- ✦ TCP Connection Management Modeling
- ✦ TCP Sliding Window
- ✦ TCP Timer Management
- ✦ TCP Congestion Control

## 6.5.1 Introduction to TCP

- TCP: **T**ransmission **C**ontrol **P**rotocol      传输控制协议
  - ✦ Connection-oriented
  - ✦ It **does almost everything**: reliable, end-to-end, byte stream
  - ✦ RFC 9293, Aug. 2022. ( obsoletes: 793, 879, 2873, 6093, 6429, 6528, 6691, etc.)
- Characters
  - ✦ Data **S**tream Transfer      数据流传输
  - ✦ Full Duplex      全双工
  - ✦ Multiplexing      复用
  - ✦ Logical Connections      逻辑连接
  - ✦ Reliability      可靠性
  - ✦ Flow Control      流量控制



## 6.5.2 The TCP Service Model

### ➤ Socket 套接口

- ✦ has a **socket number** (address), consisting of the **IP address** of the host, and a 16-bit number local to that host, called a **port**.
  - A **port** (端口) is the TCP name for a **TSAP**.
  - **well-known ports**: Port numbers **below 1024** are reserved for **standard services** that can usually only be started by **privileged** users.
- ✦ A socket may be used for **multiple connections** at the same time.

### ➤ Connections 连接

- ✦ 5 tuple 五元组
- ✦ identified by the **protocol** (TCP),
- ✦ and the **socket identifiers** (**socket descriptor**, 套接口描述字, 套接字) at **both ends**, that is, (**socket1**, **socket2**),

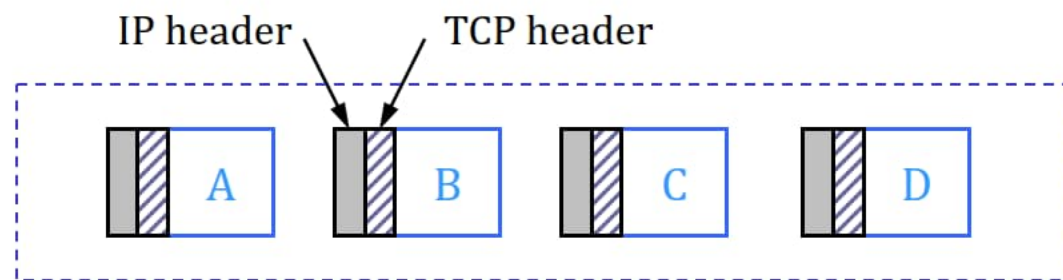
# Well-known Ports



| Port | Protocol | Use                            |
|------|----------|--------------------------------|
| 21   | FTP      | File transfer                  |
| 22   | SSH      | Secure Shell                   |
| 23   | Telnet   | Remote login                   |
| 25   | SMTP     | E-mail                         |
| 69   | TFTP     | Trivial File Transfer Protocol |
| 79   | Finger   | Lookup info about a user       |
| 80   | HTTP     | World Wide Web                 |
| 110  | POP-3    | Remote e-mail access           |
| 119  | NNTP     | USENET news                    |

Figure 6-34. Some assigned ports.

# byte stream, not message stream



(a) Four 512-byte segments sent as separate IP datagrams.



(b) The 2048 bytes of data delivered to the application in a single READ call.

Figure 6-35.

## 6.5.3 The TCP Protocol

- SN (Sequence Number)
  - ✦ every byte on a TCP connection has its own 32-bit sequence number.
- TPDU (TCP Protocol Data Unit)
  - ✦ A TCP segment consists of a fixed 20-byte header (plus an optional part) followed by zero or more data bytes. 段
- Two limits restrict the segment size. 最大段长
  - ✦ each segment, including the TCP header, must fit in the 65,515 bytes IP payload.  
(65,535-20=65515)
  - ✦ each link has an MTU (Maximum Transfer Unit).
    - the MTU is generally 1500 bytes (the Ethernet payload size).



## 6.5.4 The TCP Segment Header

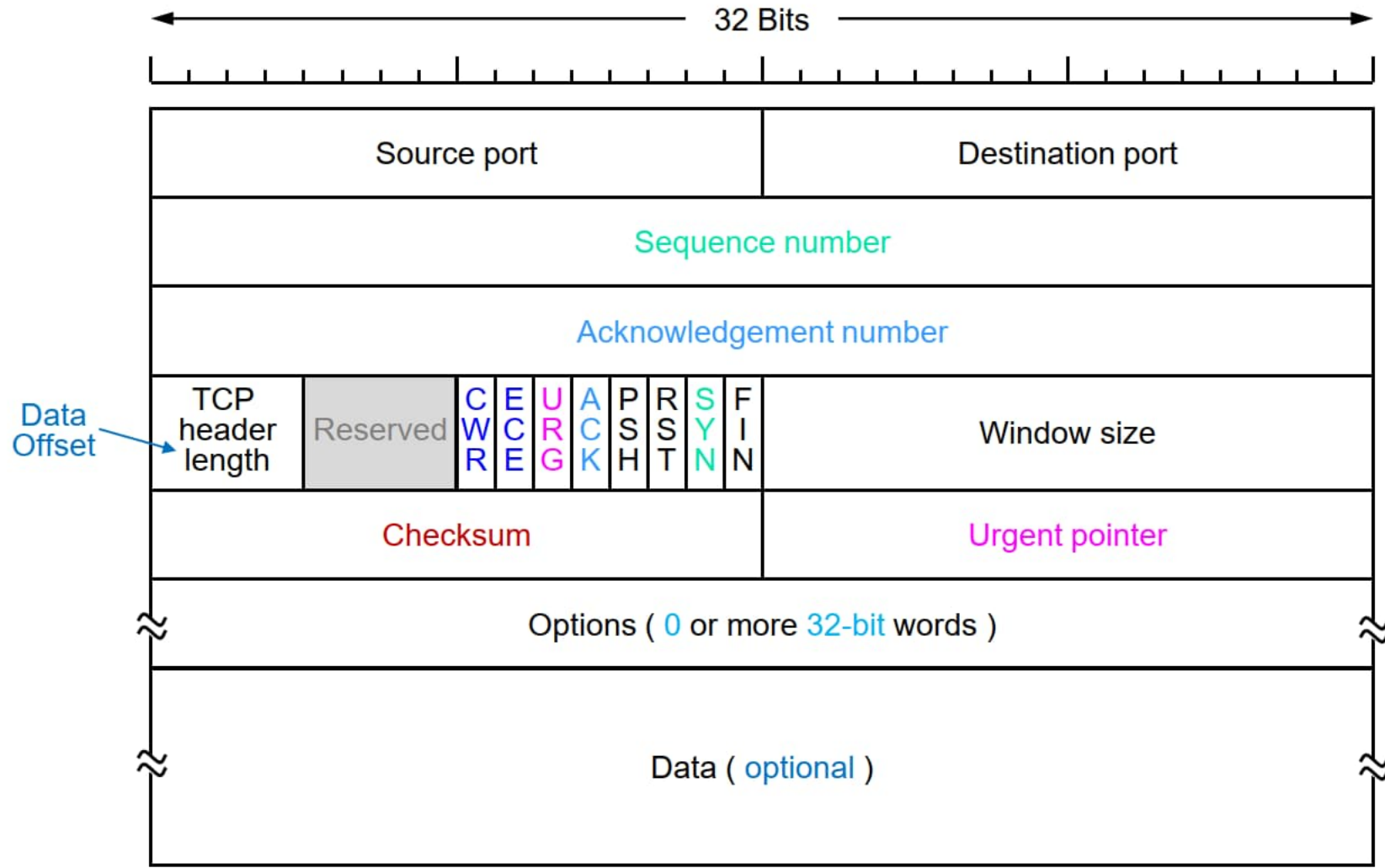
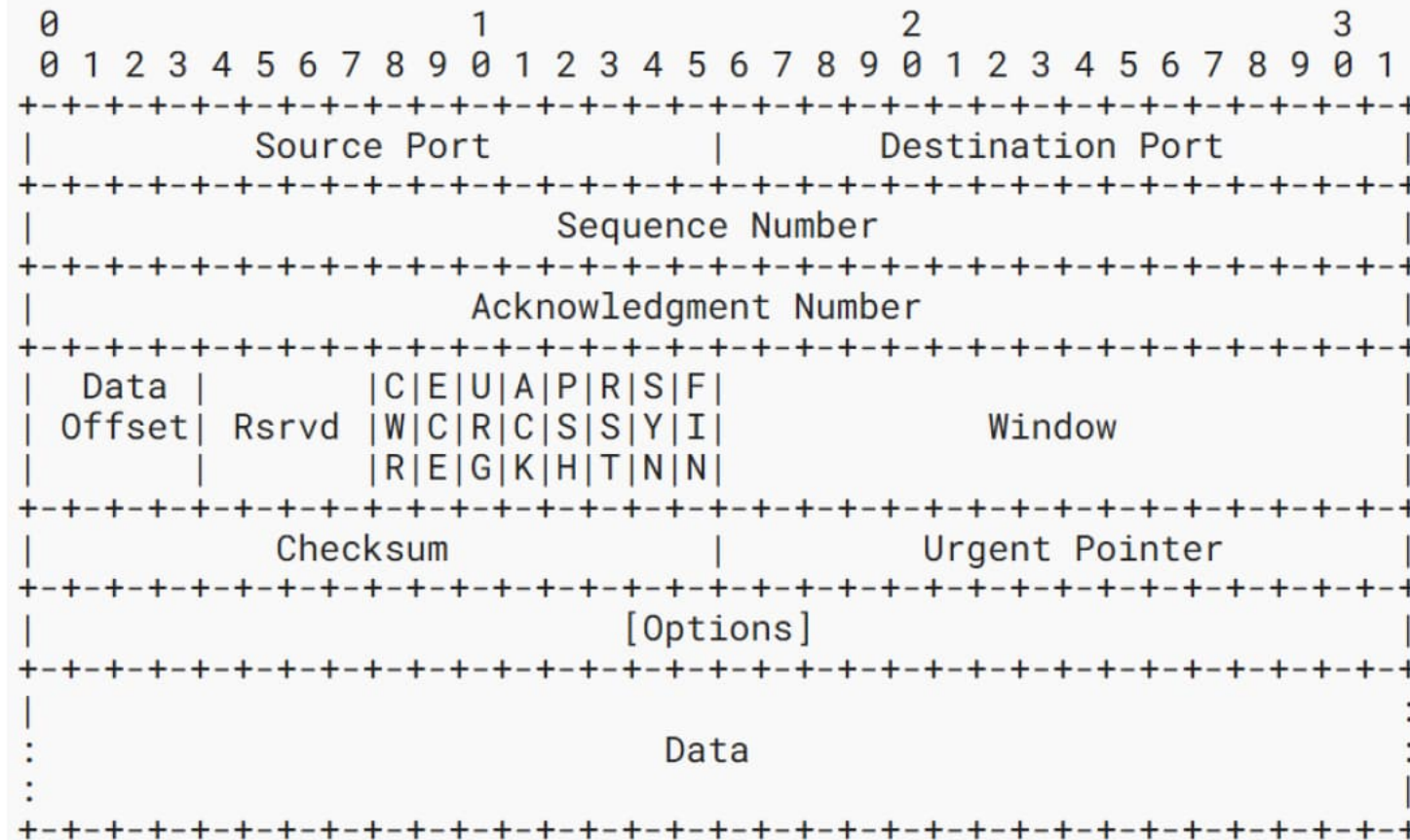


Figure 6-36. The TCP header.

# TCP Header Format from RFC9293

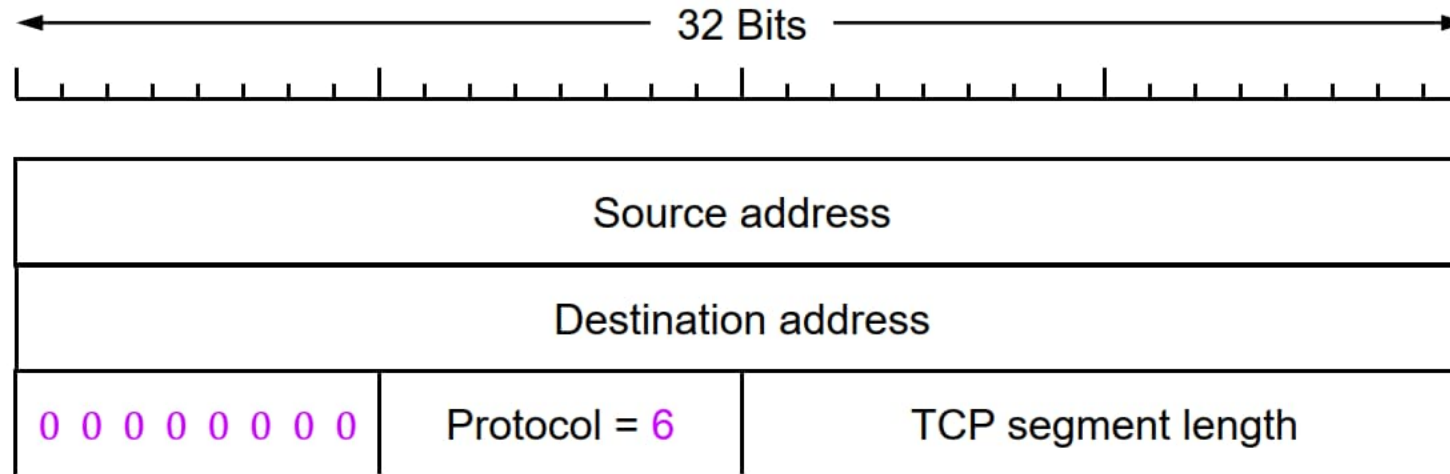


TCP Header Format from RFC9293

# Control bits

- Control bits: The control bits are also known as "flags".
  - ✦ CWR: Congestion Window Reduced.
  - ✦ ECE: ECN-Echo.
  - ✦ URG: Urgent pointer field is significant.
    - Urgent pointer: point to the last byte of this urgent data.
  - ✦ ACK: Acknowledgment field is significant.
  - ✦ PSH: Push function.
  - ✦ RST: Reset the connection.
  - ✦ SYN: Synchronize sequence numbers.
  - ✦ FIN: No more data from sender.

# The IPv4 pseudo-header



The pseudo IP header included in the TCP checksum.



# TCP Options

## ➤ Option length

- ✦ variable length, fill a multiple of 32 bits by using padding with zeros,
- ✦ may extend to 40 bytes.  $((2^4 - 1) - 5) \times 4$

## ➤ Encoding:

- ✦ TLV: Type-Length-Value 类型-长度-值

## ➤ Some Options

- ✦ MSS: Maximum Segment Size 最大段长度
  - All hosts are required to accept TCP segments of  $536 + 20 = 556$  bytes.
- ✦ Timestamp 时间戳
  - used to compute round-trip time
  - PAWS: Protection Against Wrapped Sequence Numbers 序号环绕保护
- ✦ SACK: Selective ACKnowledgement 选择性确认

# 6.5.5 TCP Connection Establishment

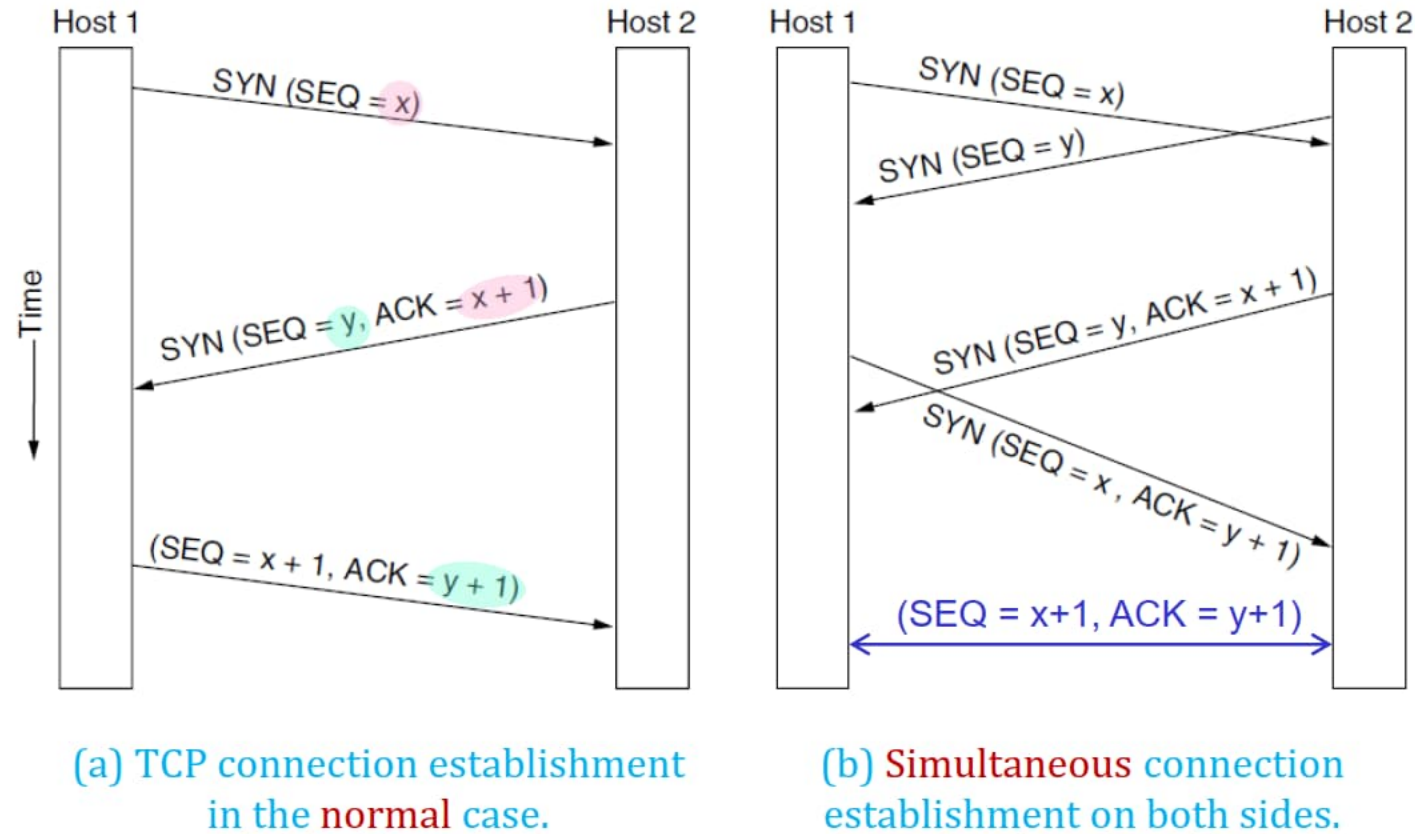
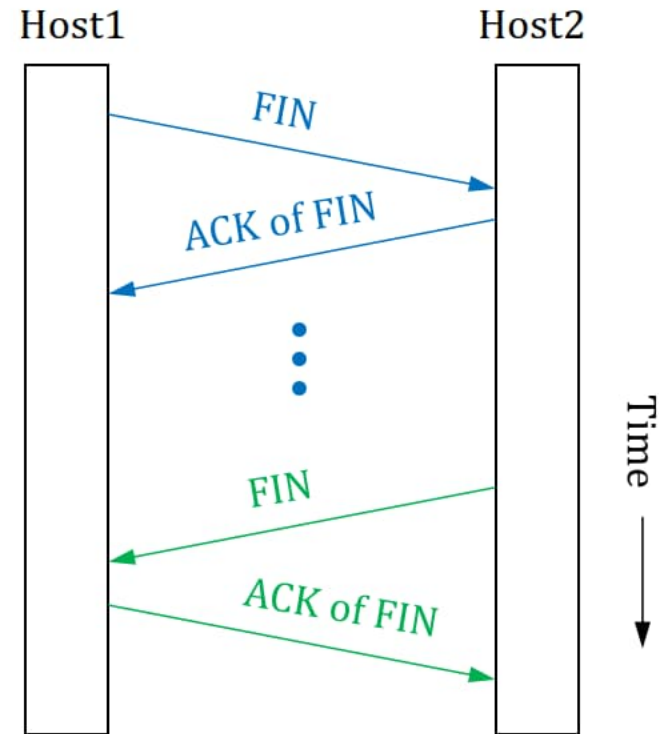


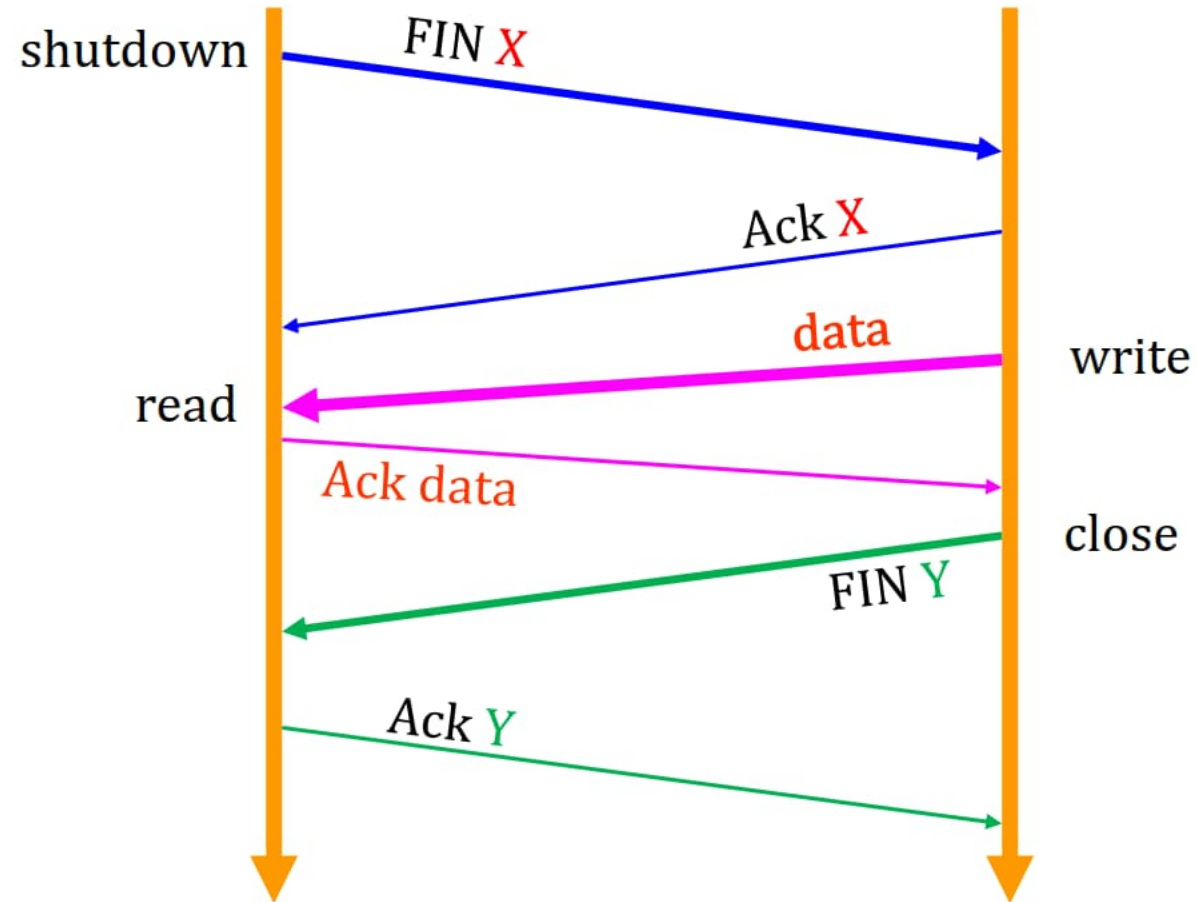
Figure 6-37.

connections are identified by their end points,  
just one connection is established

## 6.5.6 TCP Connection Release



# half closing





## 6.5.7 TCP Connection Management Modeling

| State       | Description                                             |
|-------------|---------------------------------------------------------|
| CLOSED      | No connection is active or pending                      |
| LISTEN      | The server is waiting for an incoming call              |
| SYN_RCVD    | A connection request has arrived; wait for ACK          |
| SYN_SENT    | The application has <b>started to open</b> a connection |
| ESTABLISHED | The normal data transfer state                          |
| FIN_WAIT_1  | The <b>application</b> has said it is finished          |
| FIN_WAIT_2  | The other side has <b>agreed to</b> release             |
| TIME_WAIT   | Wait for <b>all packets</b> to <b>die off</b>           |
| CLOSING     | <b>Both sides</b> have tried to close simultaneously    |
| CLOSE_WAIT  | The <b>other side</b> has initiated a release           |
| LAST ACK    | Wait for <b>all packets</b> to <b>die off</b>           |

Figure 6-38. The states used in the TCP connection management **finite state machine**.

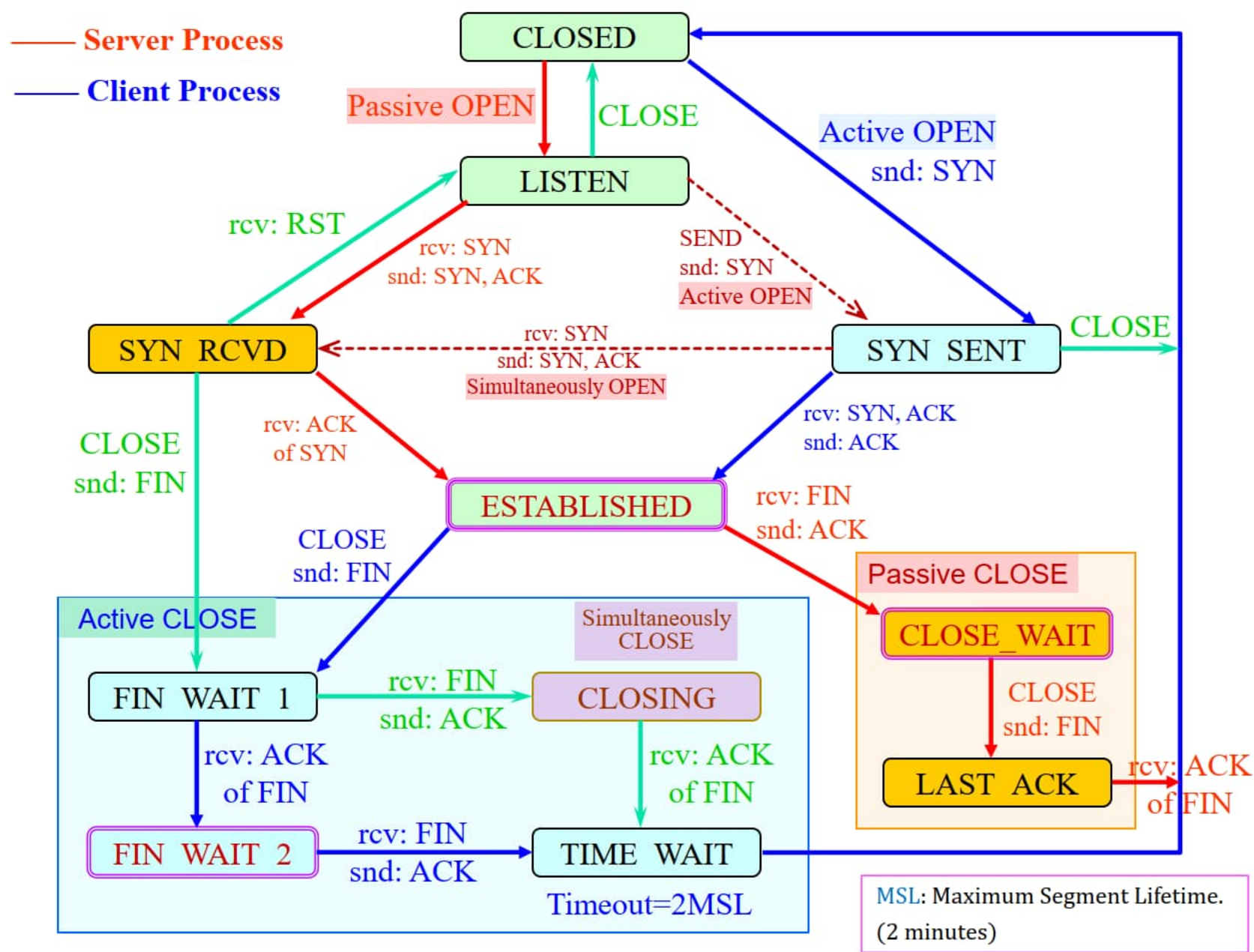
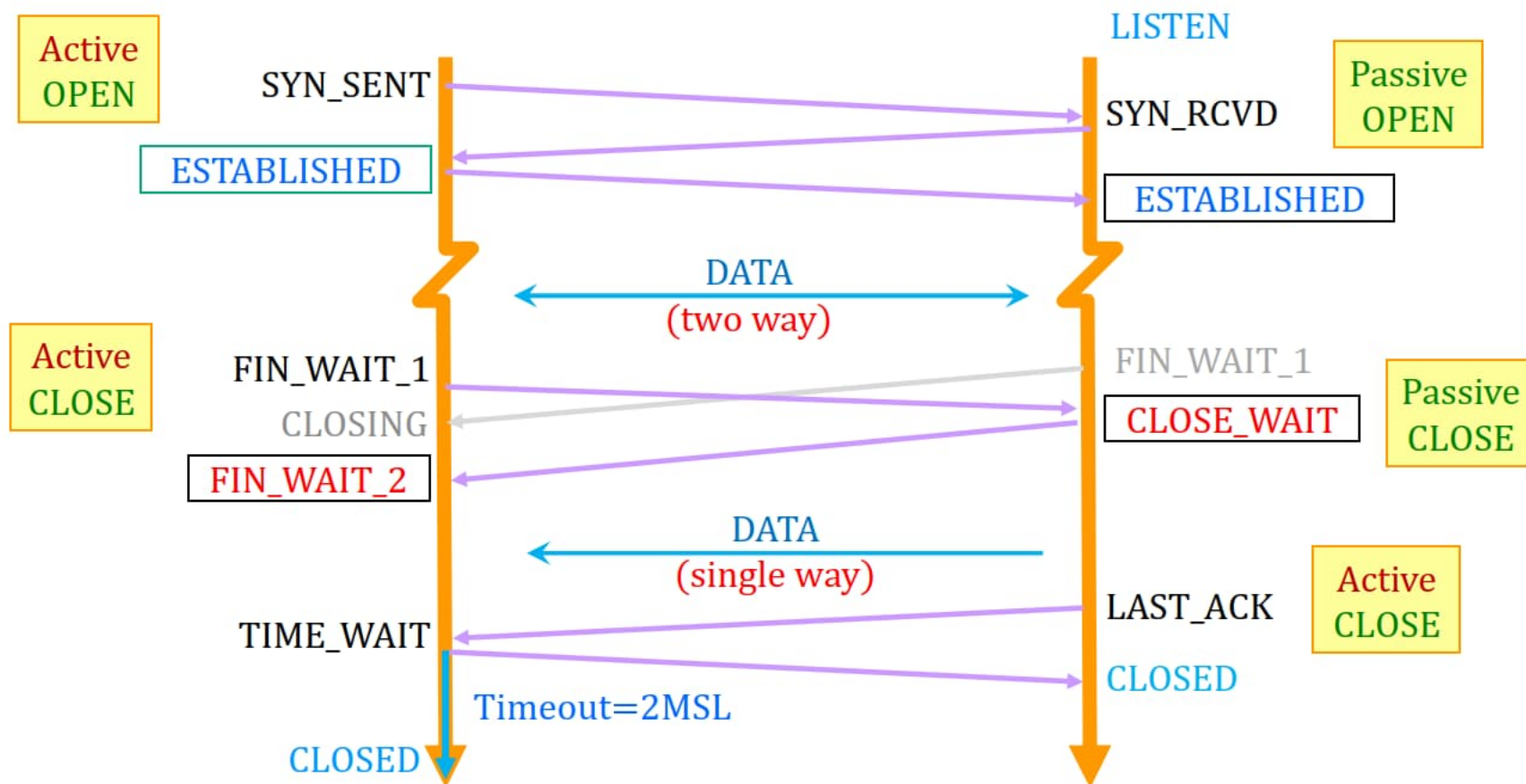


Figure 6-39. A summary of TCP connection management finite state machine.

# TCP Timing Diagram



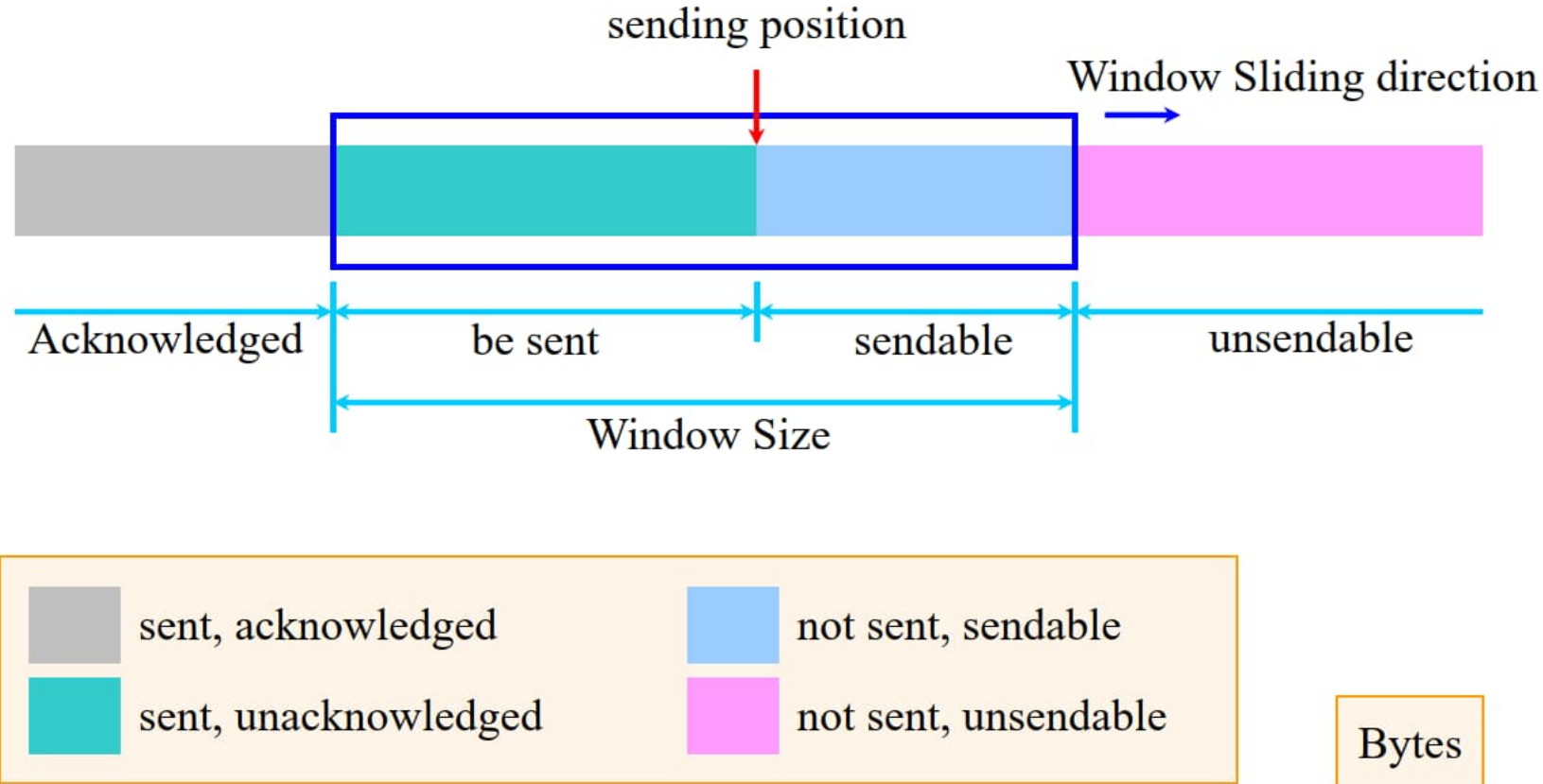
TCP时序图



## 6.5.8 TCP Sliding Window



### ➤ Window Size vs. Flow Control





# Window management

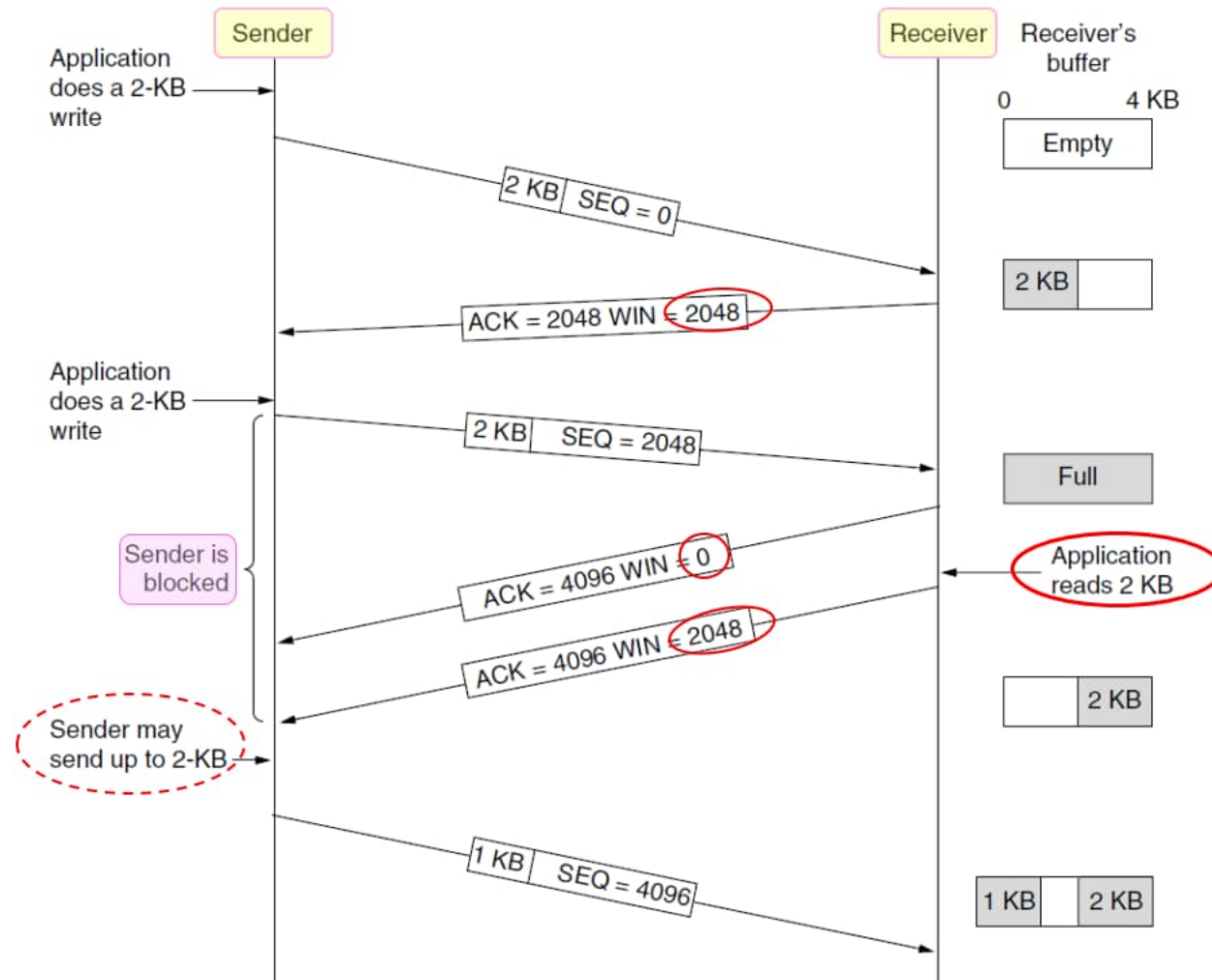
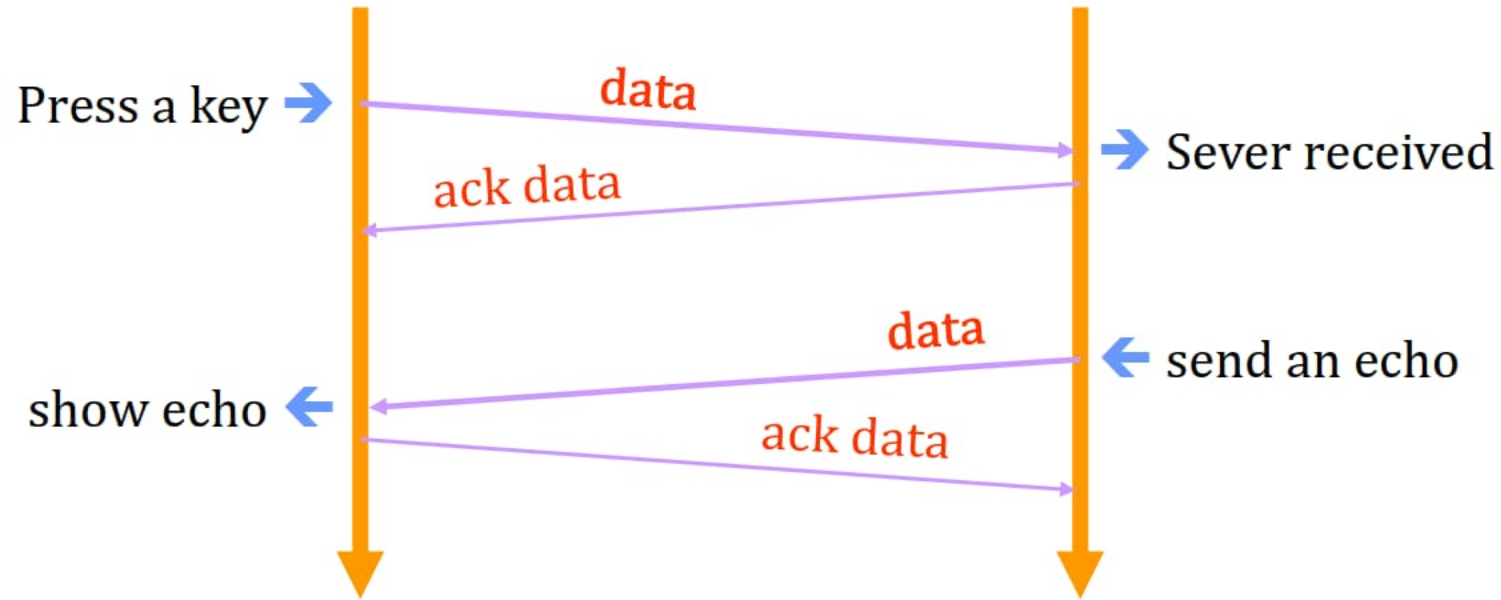


Figure 6-40. Window management in TCP.

# interactive data



$$1 / (20 + 20 + 1) = 2.44\%$$

Block: 50% packets & 90% data

Interactive: 50% packets & 10% data

# Nagle's algorithm



## ➤ Policy

Sender

- ✦ send the first piece,
- ✦ buffer all the rest, until the first piece is acknowledged.

## ➤ Benefit

- ✦ Nagle's algorithm will put the many pieces in one segment, greatly reducing the bandwidth used.

## ➤ Problem

- ✦ in interactive games
- ✦ sometimes interact with delayed acknowledgements to cause a temporary deadlock

## ➤ Disable Nagle's algorithm

- ✦ `TCP_NODELAY` option

# Silly window syndrome

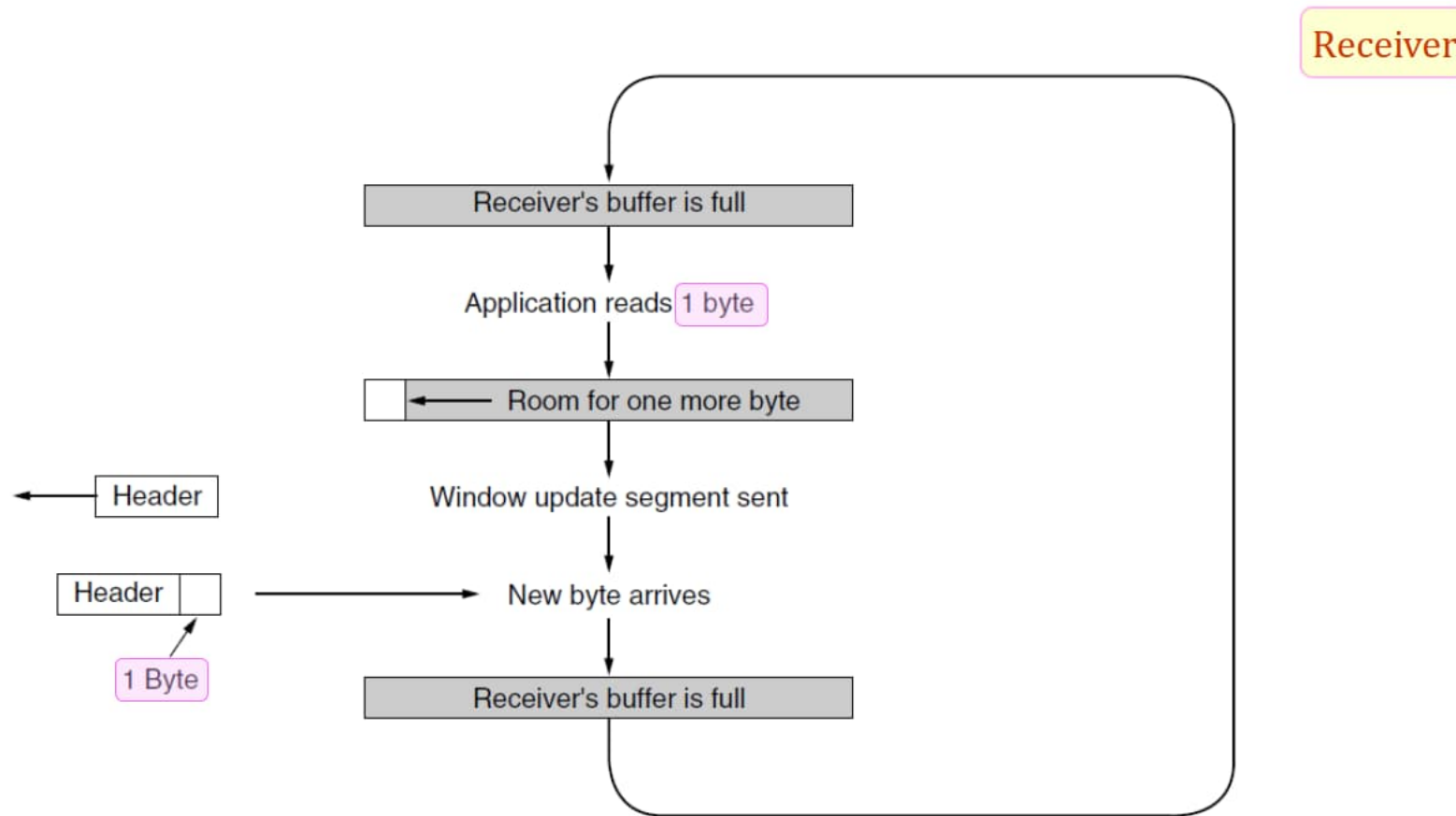


Figure 6-41. Silly window syndrome.



# Clark's Solution

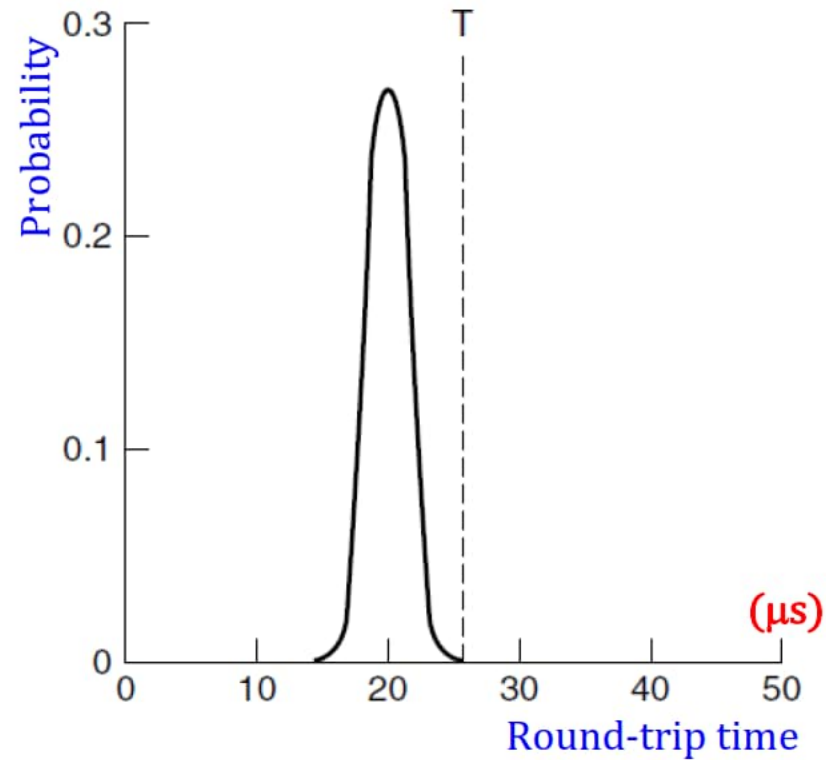


## ➤ Solution

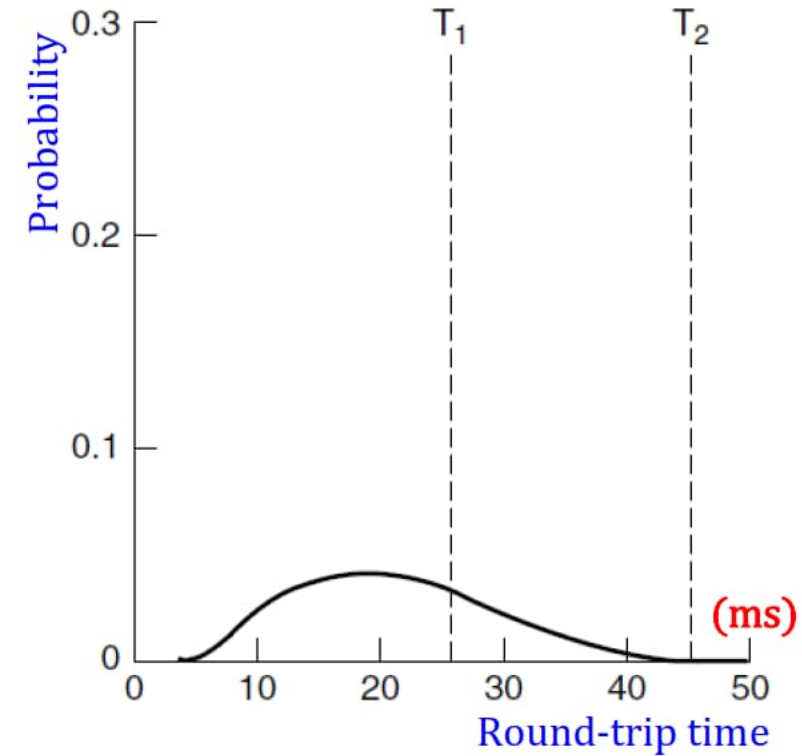
- ✦ to **prevent** the **receiver** from sending a **window update** for **1 byte**.
- ✦ it should **wait** until it can send **a full segment**, or at least one containing **half of the receiver's buffer size**.

## 6.5.9 TCP Timer Management

➤ RTT: Round-trip time 往返时间



(a) Probability density of acknowledgement arrival times in the data link layer.



(b) Probability density of acknowledgement arrival times for TCP.

Figure 6-42.

# Retransmission Timer



重传定时器

- RTO: Retransmission TimeOut TCP重传间隔
  - ✦ When a segment is sent, a retransmission timer is started.
    - If the segment is acknowledged before the timer expires, the timer is stopped.
    - If the timer goes off before the acknowledgement comes in, the segment is retransmitted.
- Question: how long should the timeout be?
  - ✦ If the timeout is set too short, unnecessary retransmissions will occur.
  - ✦ If it is set too long, performance will suffer due to the long retransmission delay whenever a packet is lost.
- Solution
  - ✦ to use a dynamic algorithm that constantly adapts the timeout interval, based on continuous measurements of network performance.

# Estimate of RTO



- **SRTT: Smoothed Round-Trip Time** 平滑往返时间
  - ✦ the best current estimate of the round-trip time to the destination
- **EWMA: Exponentially Weighted Moving Average**
  - ✦ 指数加权移动平均
  - ✦  $SRTT = \alpha \cdot SRTT + (1 - \alpha) \cdot R$ 
    - $R$ : the measurement value of **current RTT**.
    - $\alpha$ : a **smoothing factor** that determines how quickly the old values are forgotten.  
Typically,  $\alpha = 7/8$ .
- The **retransmission timeout**:
  - ✦  $RTO = SRTT + SRTT$



## ➤ Problem

- ✦ when the load approaches capacity, the delay becomes large and highly variable.

## ➤ Jacobson's Algorithm

- ✦ RTTVAR: Round-Trip Time VARIation 往返时间变化量

- $RTTVAR = \beta \cdot RTTVAR + (1 - \beta) \cdot |SRTT - R|$

- $\beta = 3/4$

- The retransmission timeout:  $RTO = SRTT + 4 \times RTTVAR$

## ➤ Karn's algorithm

- ✦ do not update estimates on any segments that have been retransmitted.
- ✦ Additionally, the timeout is doubled on each successive retransmission until the segments get through the first time.

# Other timers



## ➤ Persistence timer 持续定时器

- ✦ to prevent the deadlock,
- ✦ when the receiver sends an acknowledgement with a window size of 0.

## ➤ Keepalive timer 保持存活定时器

- ✦ to cause one side to check whether the other side is still there,
- ✦ when a connection has been idle for a long time.

## ➤ the timer used in the TIME WAIT state

- ✦ twice the maximum packet lifetime.
- ✦ to make sure that when a connection is closed, all packets created by it have died off.

## 6.5.10 TCP Congestion Control

- Congestion control window 拥塞窗口
  - ✦ the number of bytes the sender may have in the network at any time.
- Flow control window 流量窗口
  - ✦ the number of bytes that the receiver can buffer.
- the number of bytes that may be sent
  - ✦ is the smaller of the two windows.

# Slow start

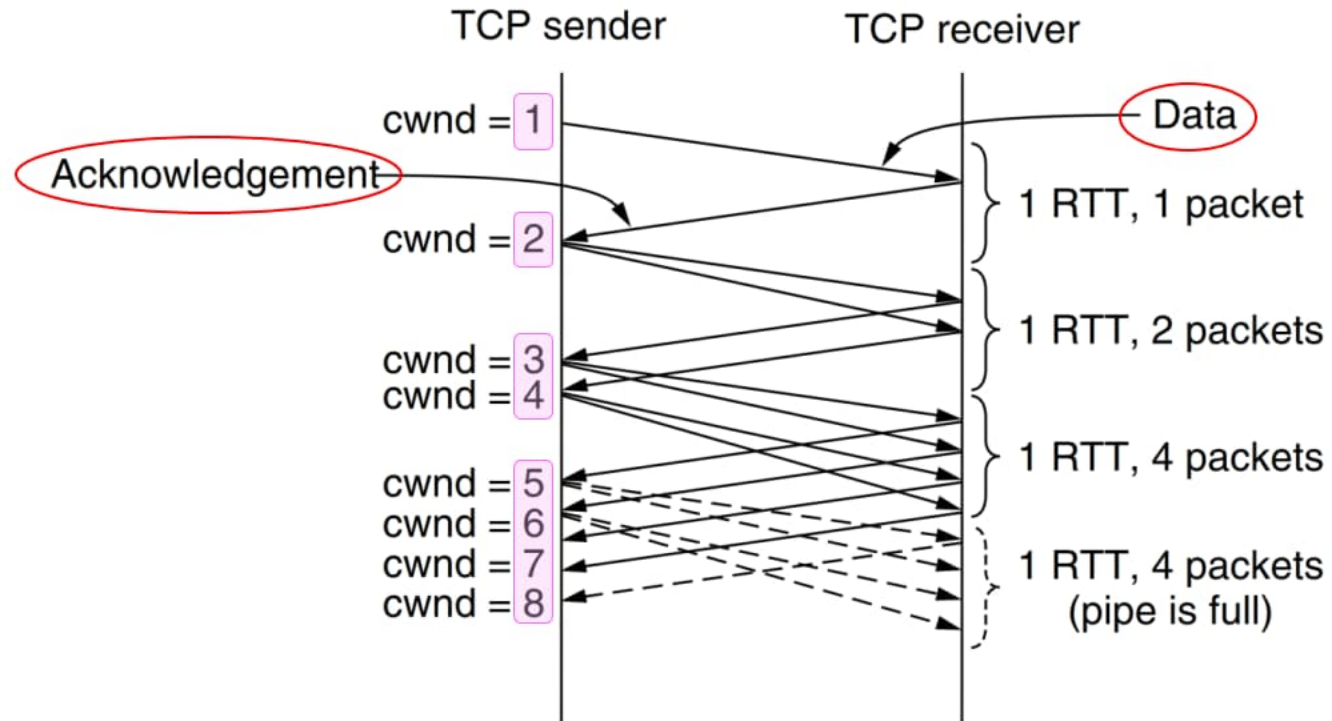


Figure 6-44. Slow start from an initial congestion window of one segment.



# Additive increase

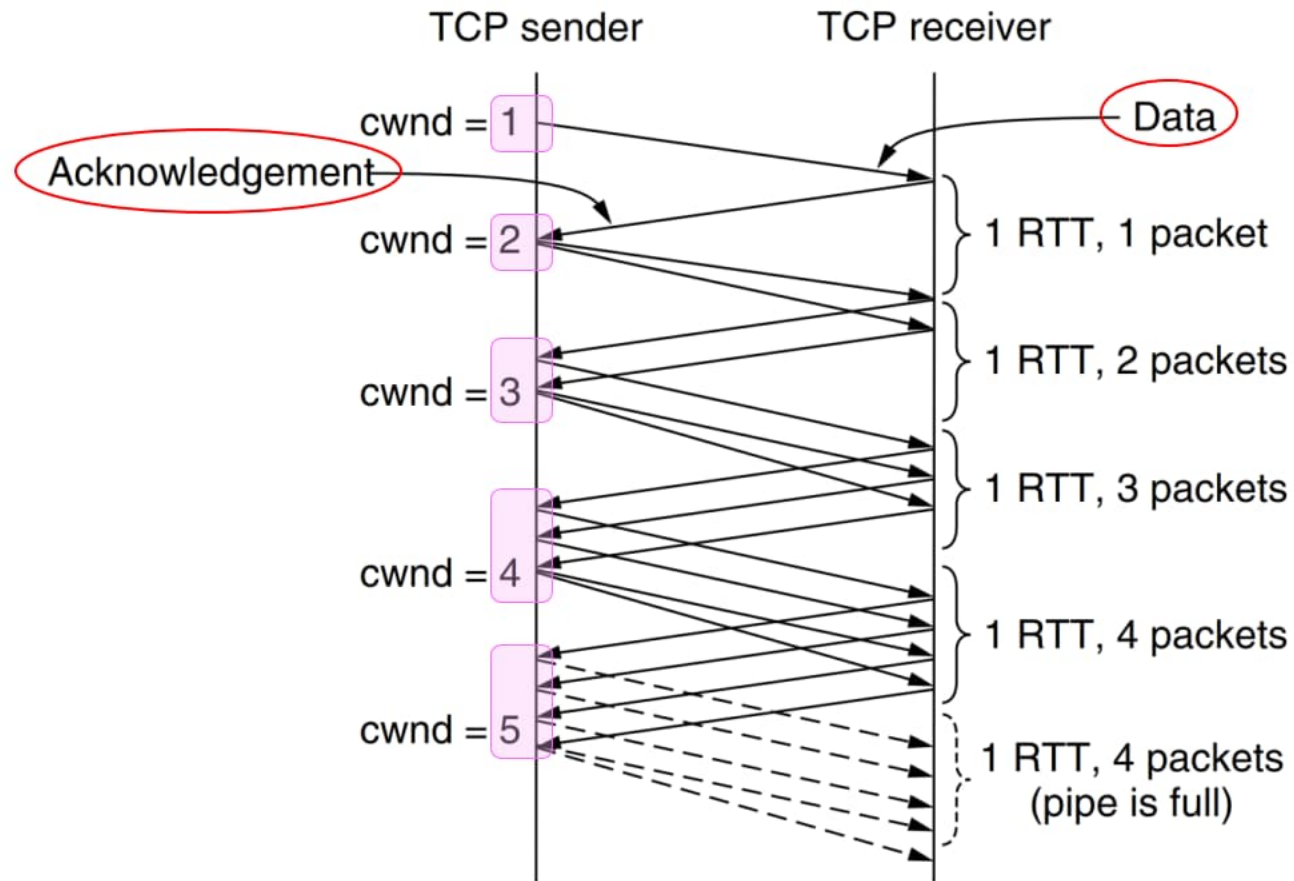


Figure 6-45. Additive increase from an initial congestion window of one segment.

# TCP Tahoe

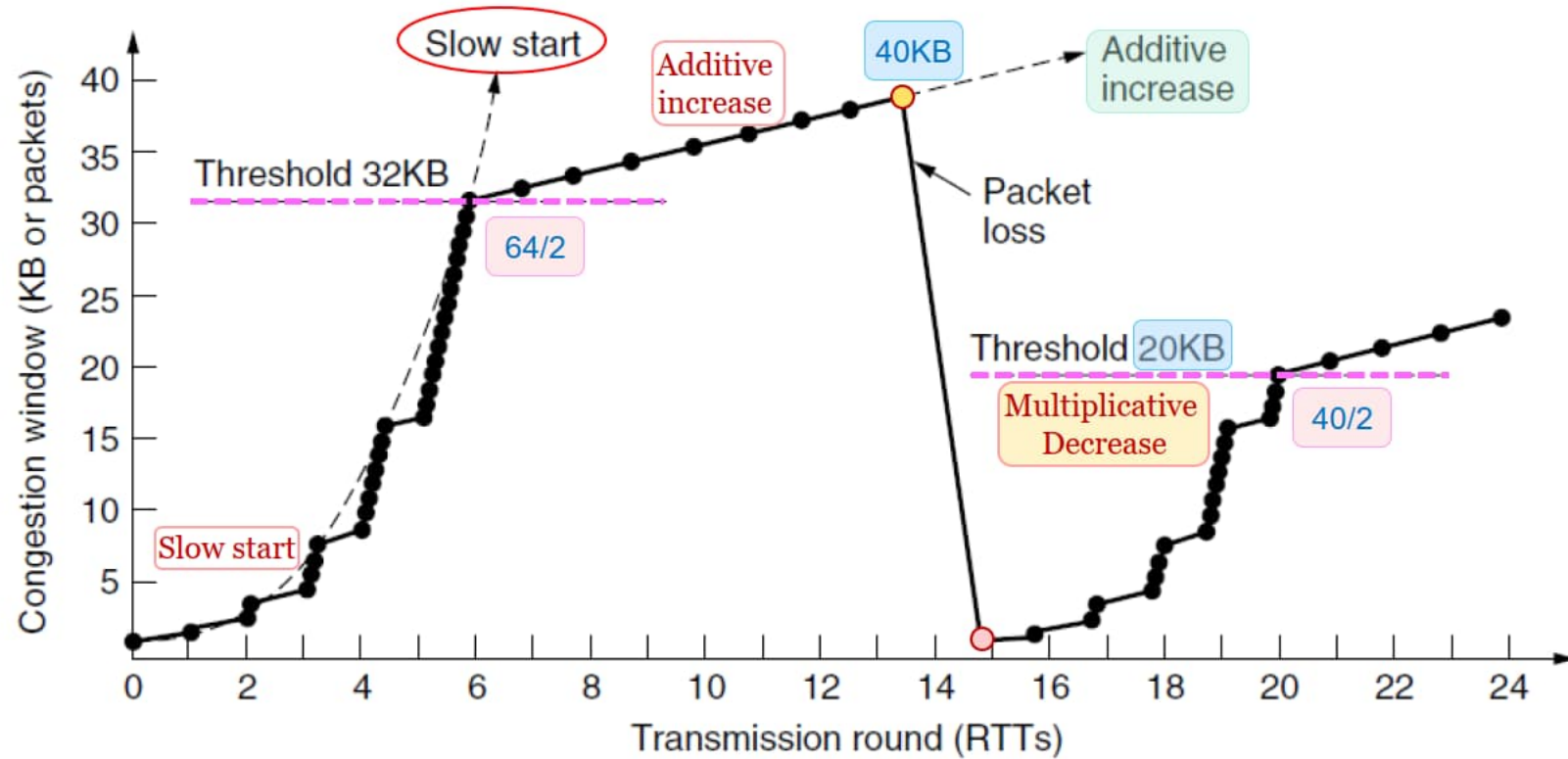


Figure 6-46. Slow start followed by additive increase in TCP Tahoe.

# Fast retransmission & Fast recovery

- Fast retransmission 快重传
  - ✦ Three duplicate acknowledgements 3次重复的确认
    - imply that a packet has been lost.
  - ✦ This packet can then be retransmitted right away, 立即重传
    - Before the retransmission timeout fires.
- Fast recovery 快恢复
  - ✦ half the value of the congestion window at the time of the fast retransmission. 拥塞窗口减半

# TCP Reno



AIMD: Additive Increase Multiplicative Decrease

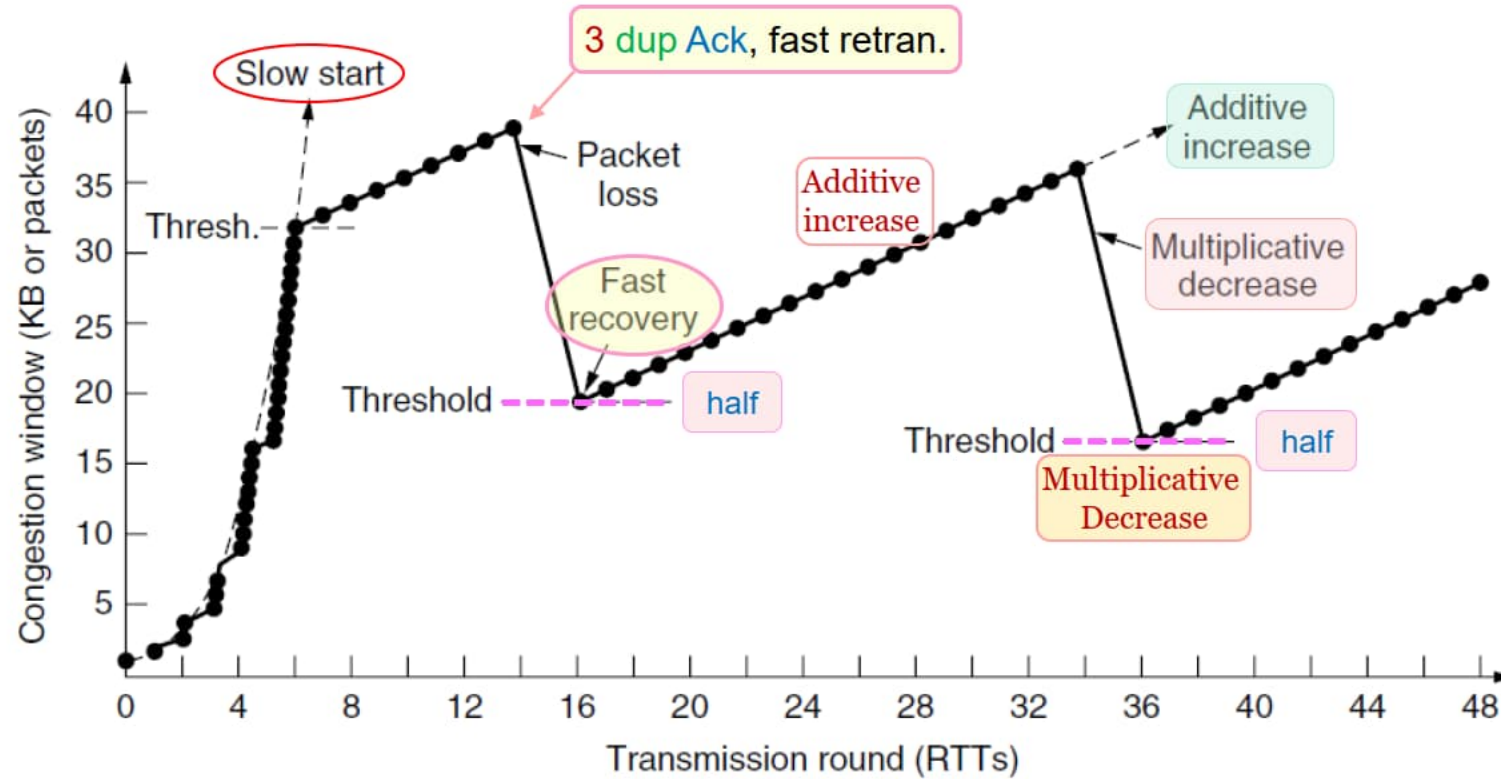


Figure 6-47. Fast recovery and the sawtooth pattern of TCP Reno.



# Selective ACKnowledgements

- SACK: Selective ACKnowledgements 选择性确认

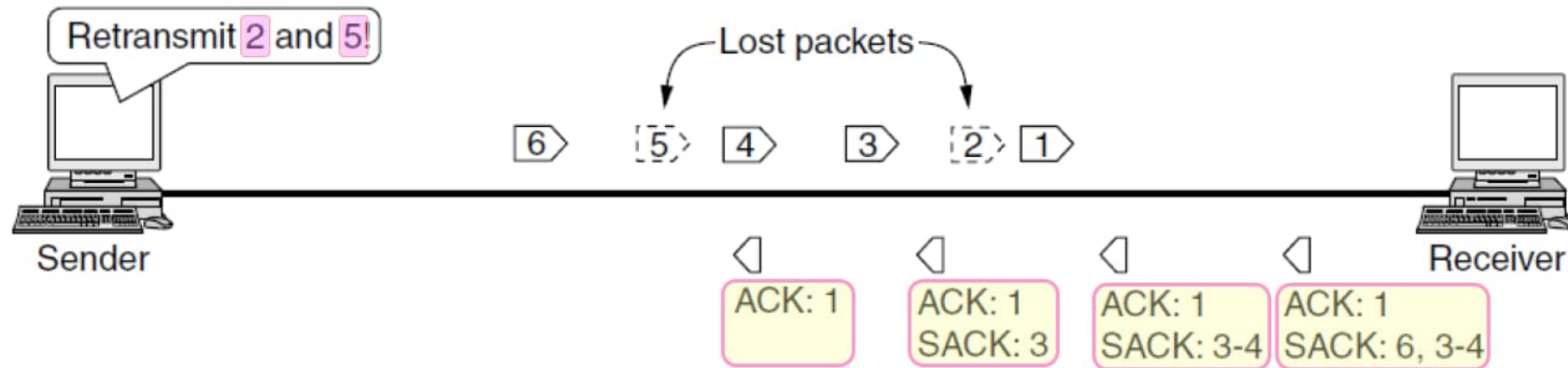


Figure 6-48. Selective acknowledgements.

# Explicit Congestion Notification



- **ECN**: Explicit Congestion Notification
  - ✦ an **IP layer** mechanism to notify hosts of congestion.
- ECN congestion signal
  - ✦ **ECE (ECN Echo)**
    - to **signal the TCP sender** that its packets have experienced **congestion**.
  - ✦ **CWR (Congestion Window Reduced)**
    - tells the **receiver** that it has **heard** the signal

## 6.7 Delay-Tolerant Networking

- DTN 容滞网络
  - ✦ Delay-Tolerant Network 容迟网络
  - ✦ or: Disruption-Tolerant Network 容断网络
- DTN vs. MANET

## 6.7.1 DTN Architecture

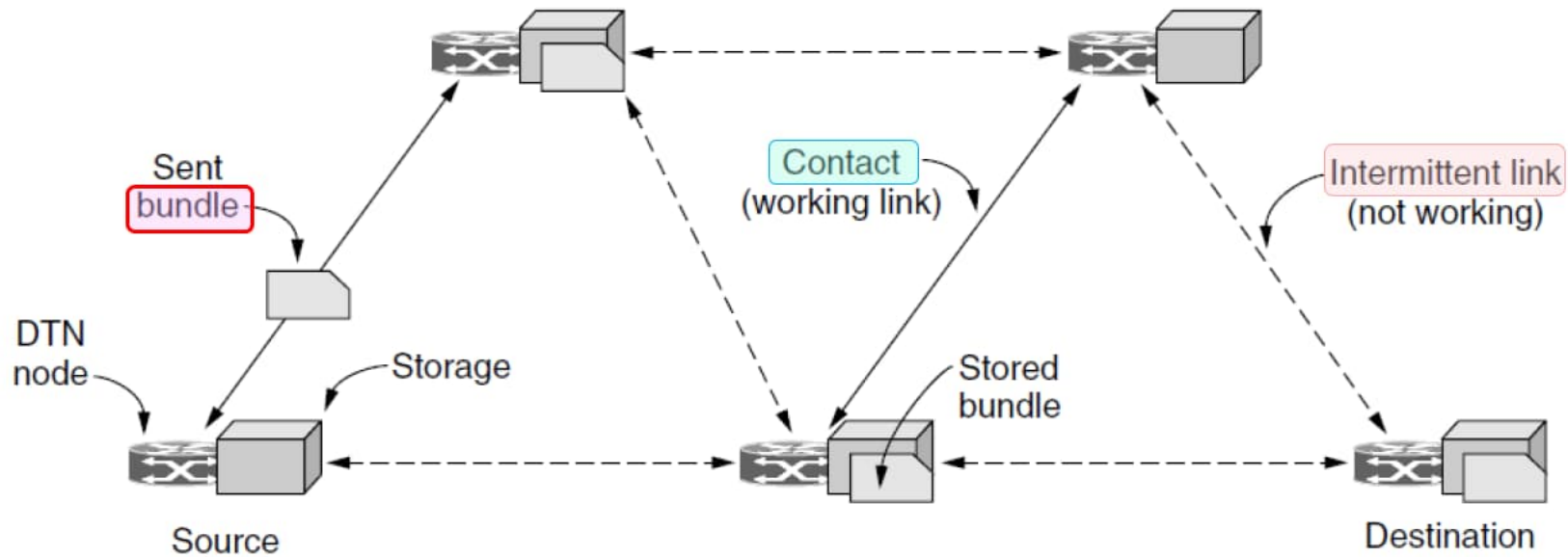
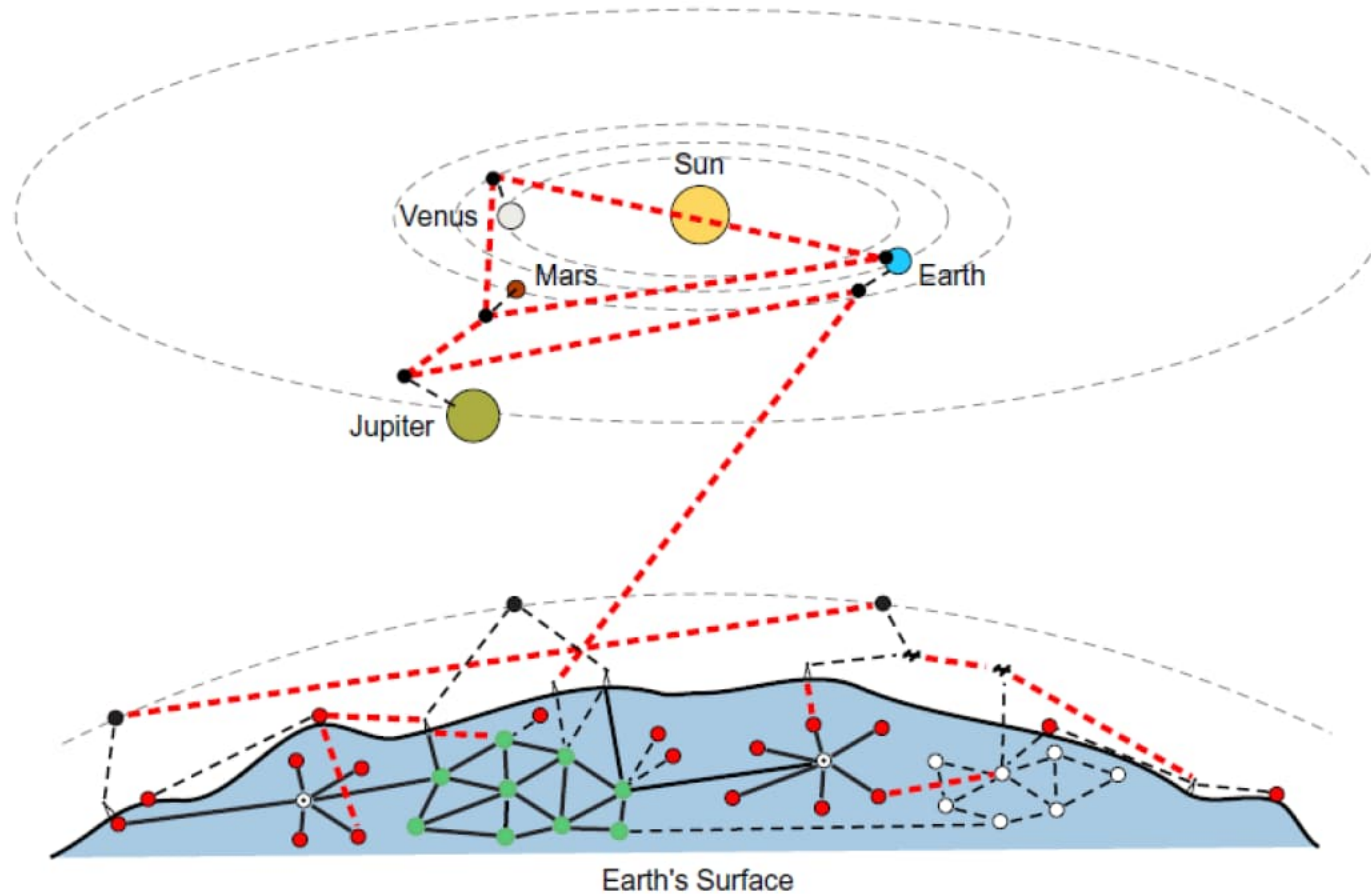


Figure 6-56. Delay-tolerant networking architecture.



# InterPlanet Network



- Key:
- Inter-region wireless link
  - - - Intra-region wireless link
  - Intra-region wired link
  - ^ Antenna
  - DTN gateway node
  - Internet router
  - ⊙ Telephone network office
  - Sensor
  - Person or vehicle

## 6.7.2 The Bundle Protocol

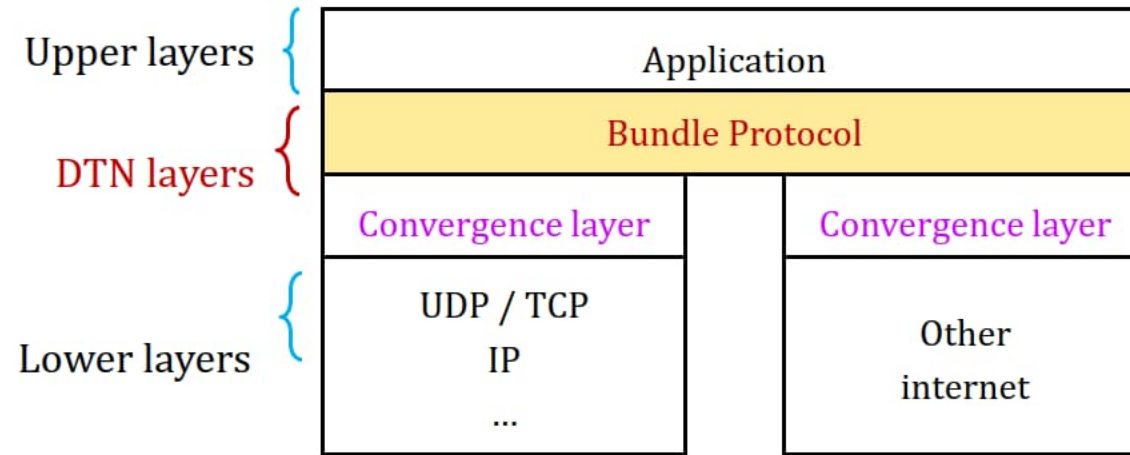
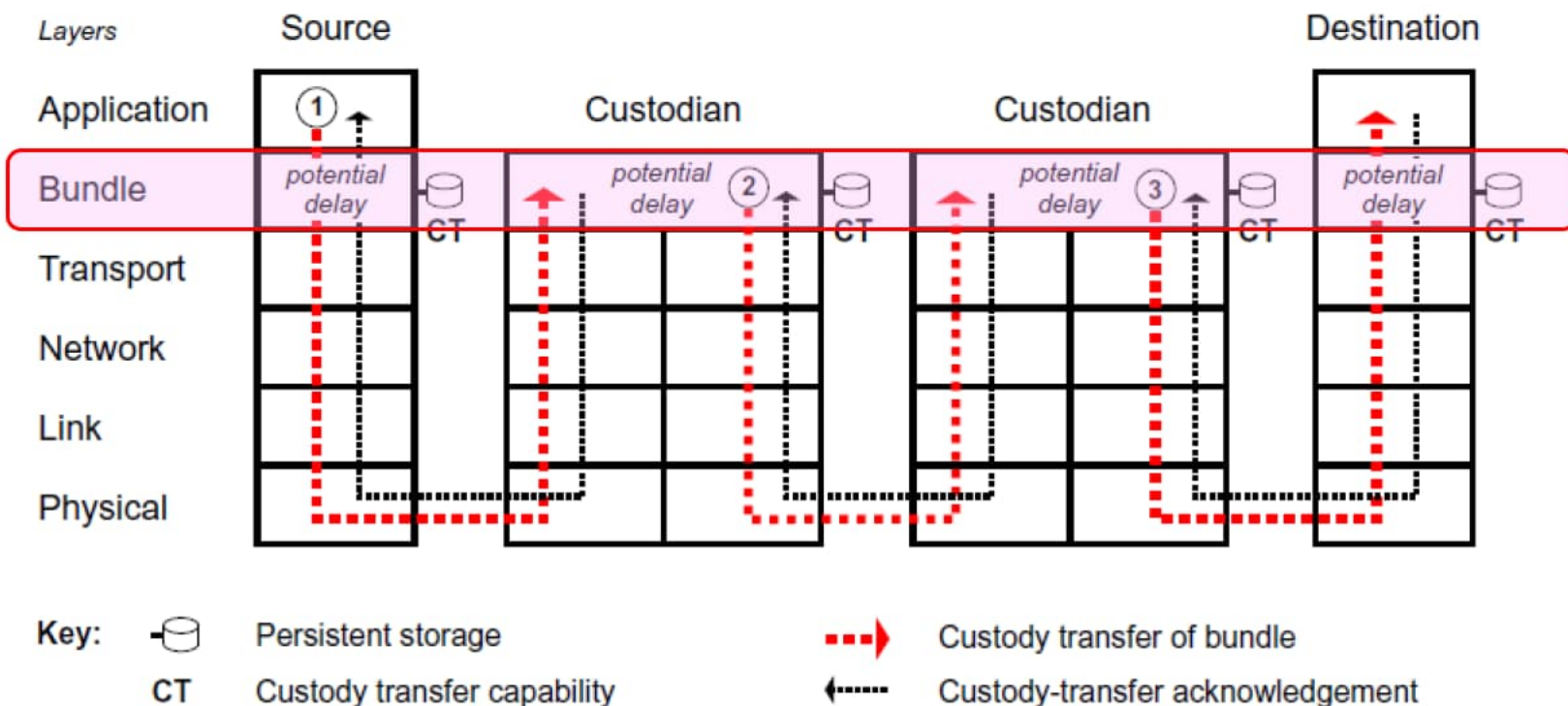


Figure 6-58. Delay-tolerant networking protocol stack.

# Custody transfer



# Summary



- The **transport layer** is the **key** to understanding **layered protocols**.
- It provides
  - ✦ various services, the **most important** of which is an **end-to-end, reliable, connection-oriented byte stream from sender to receiver**.
  - ✦ the **service primitives**, manage **connections** and **timers**, allocate bandwidth with **congestion control**, and run a variable-sized sliding window for **flow control**.
- The Internet has two main transport protocols:
  - ✦ **UDP**: **connectionless**, a **wrapper** for IP packets with the additional feature of **multiplexing** and **demultiplexing** multiple processes.
  - ✦ **TCP**: **reliable, bidirectional, congestion-controlled byte stream**,
    - using algorithms from Nagle, Clark, Jacobson, Karn, and others.
- Delay-tolerant networking
  - ✦ across networks that have **occasional connectivity** or **long delays** across links.

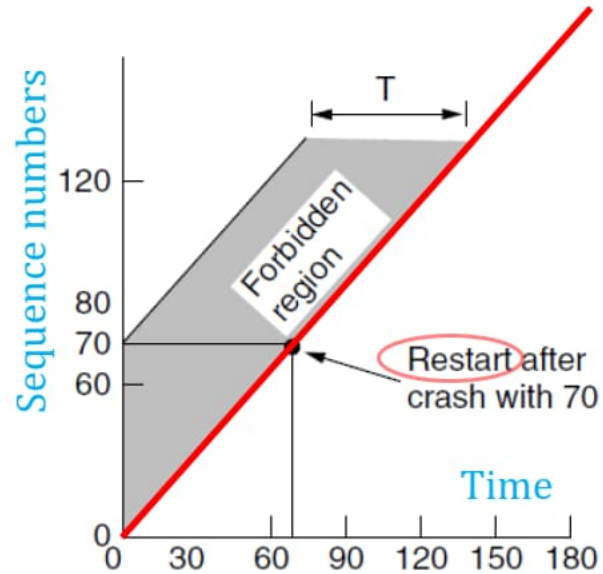




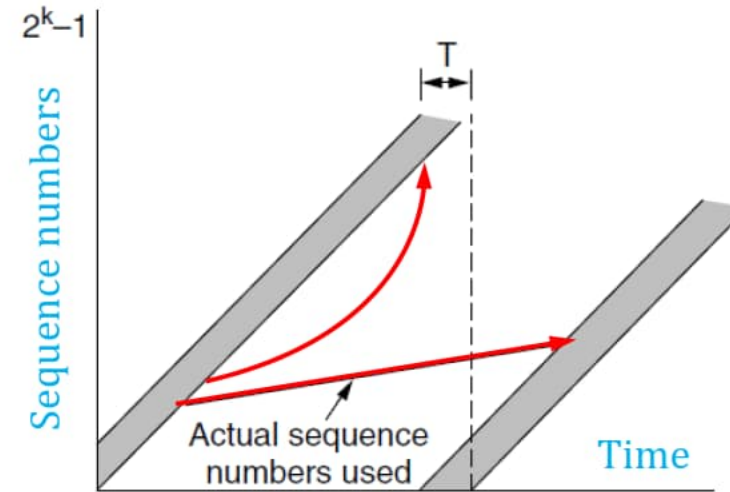




## 6.2.2 Connection Establishment



(a) Segments may not enter the forbidden region.



(b) The resynchronization problem.

Figure 6-10.



## 6.3.3 Wireless Issues

### ➤ Issue

- ✦ Packet loss is often used as a congestion signal.
- ✦ Wireless networks lose packets all the time due to transmission errors.

### ➤ Solution

- ✦ to mask the wireless losses by using retransmissions over the wireless link.
  - 802.11 uses a stop-and-wait protocol to deliver each frame.

### ➤ time scale

- ✦ Link layer retransmission
- ✦ Loss timers in transport

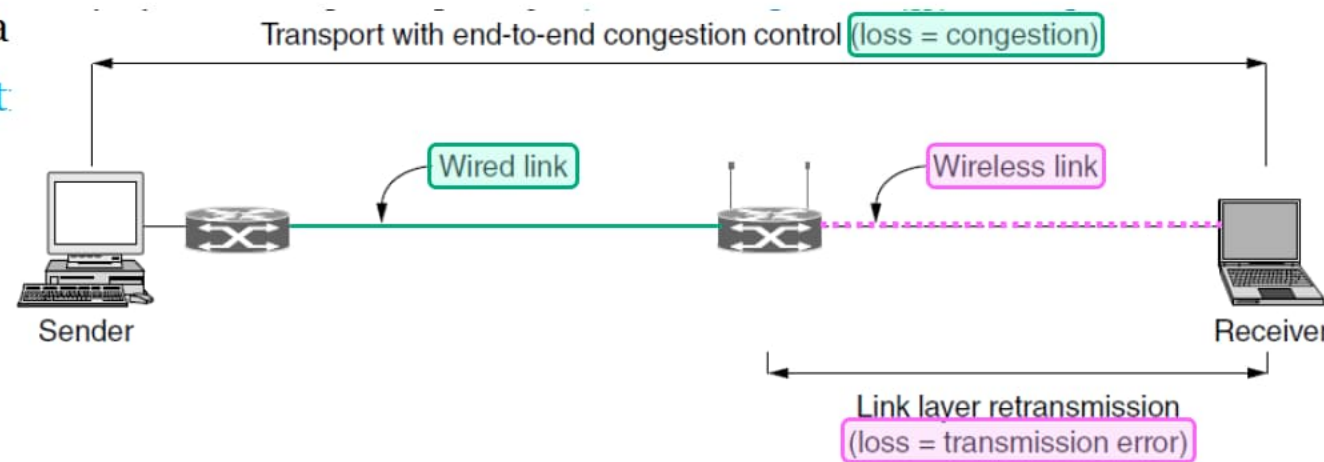


Figure 6-26. Congestion control over a path with a wireless link.

## 6.6 Performance Issues



- Six aspects of network performance
  - ✦ Performance problems.
  - ✦ Measuring network performance.
  - ✦ Host design for fast networks.
  - ✦ Fast segment processing.
  - ✦ Header compression.
  - ✦ Protocols for “long fat” networks.

## 6.6.1 Performance Problems in Computer Networks

- Congestion 拥塞
  - ✦ Temporary resource overloads 资源临时过载
  - ✦ Structural resource imbalance 结构性资源不平衡
    - CPU, memory, buffer space, bandwidth
  - ✦ Synchronously triggered overloads 由同步触发引起的过载
    - Broadcast storm 广播风暴
    - Electrical power **failure** 掉电恢复
  - ✦ Degrade 性能退化
- RTO setting 定时间隔设置不合理
- Jitter control 抖动控制
  - ✦ Real-time applications 音频、视频传输等实时应用

## 6.6.2 Network Performance Measurement



### ➤ Potential pitfalls 潜在的陷阱

- ✦ Make Sure That the Sample Size Is Large Enough
  - 确保样本空间足够大
- ✦ Make Sure That the Samples Are Representative
  - 确保样本具有代表性
- ✦ Caching Can Wreak Havoc with Measurements
  - 缓存可能破坏测量结果
- ✦ Be Sure That Nothing Unexpected Is Going On during Your Tests
  - 确保测试期间不会发生不可知的事情
- ✦ Be Careful When Using a Coarse-Grained Clock
  - 小心使用粗粒度时钟
- ✦ Be Careful about Extrapolating the Results
  - 小心推断的结果



# Be Careful about Extrapolating

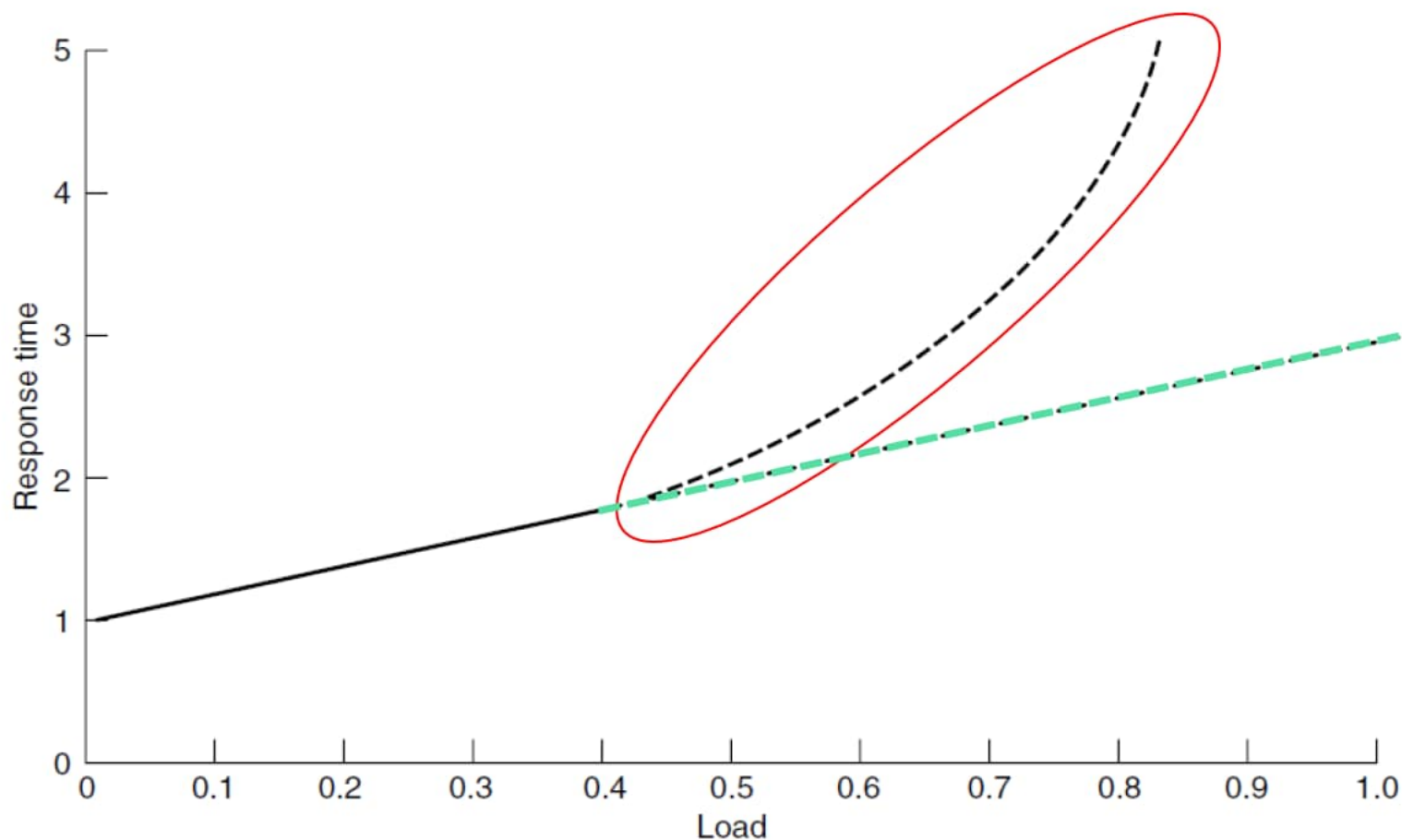


Figure 6-49. Response as a function of load.



## 6.6.3 Host Design for Fast Networks

- Host Speed Is More Important Than Network Speed
  - ✦ 主机速度比网络速度更重要
- Reduce Packet Count to Reduce Overhead
  - ✦ 减少分组数目来降低开销
- Minimize Data Touching
  - ✦ 最小化数据预读取
- Minimize Context Switches
  - ✦ 最小化上下文切换
- Avoiding Congestion Is Better Than Recovering from It
  - ✦ 避免拥塞比从拥塞中恢复更好
- Avoid Timeouts
  - ✦ 避免超时

# Minimize Context Switches

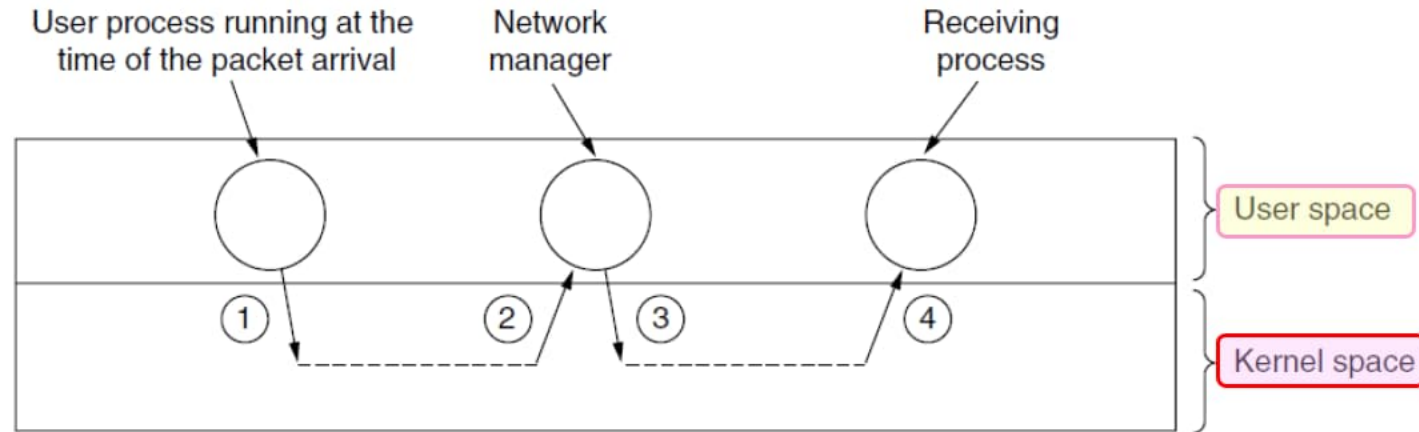


Figure 6-50. Four context switches to handle one packet with a user-space network manager.

# Message format of Bundle

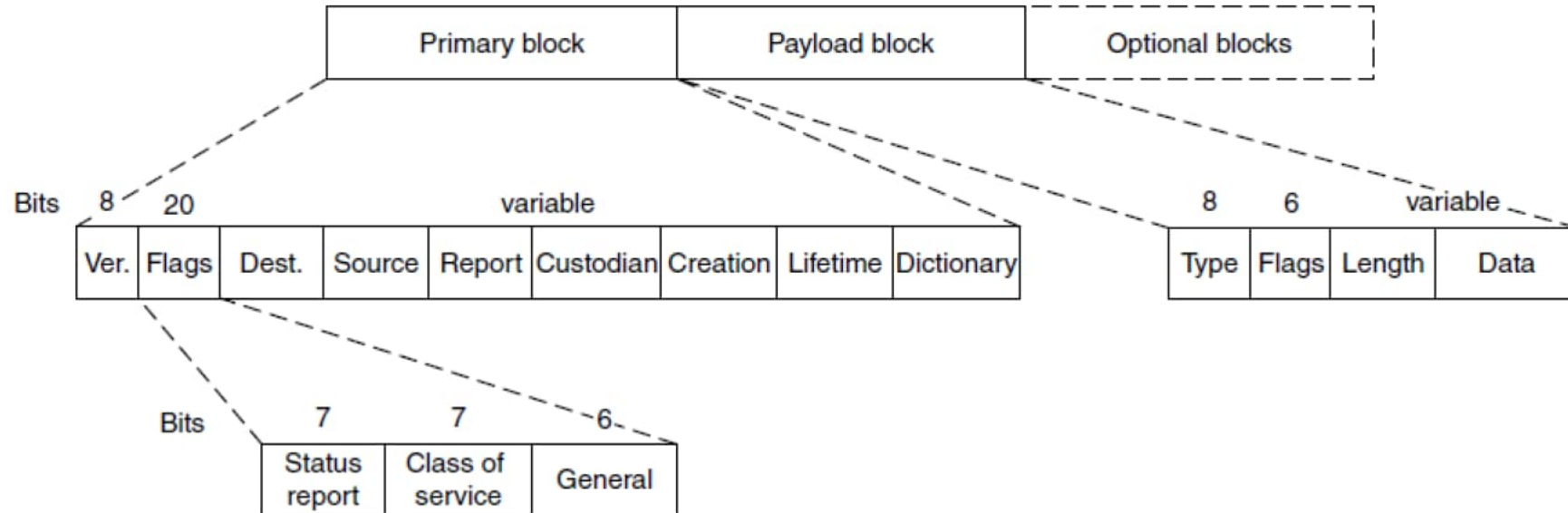


Figure 6-59. Bundle protocol message format.